VCF Operations
30-Day Mastery Guide

Complete Study Guide | VCF 9.x Aligned

Detailed Step-by-Step Edition

30-Day Mastery Plan

Interview Prep Pack (50 Questions with Exact Answers)

Hands-On Lab Workbook

Automation Starter Kit

Generated: March 21, 2026

Table of Contents

VCF OPERATIONS 30-DAY MASTERY PLAN - EXTREMELY DETAILED

VCF 9.x Aligned | Every Action Spelled Out | Zero Assumptions

WEEKLY STRUCTURE (NON-NEGOTIABLE)

WEEK 1: DAYS 1-7 (Foundation: Navigate, Inventory, Health, Dashboards Basics)

DAY 1 - VCF OPS MENTAL MODEL & UI MAP

DAY 2 - INVENTORY & OBJECT MODEL

DAY 3 - HEALTH & DIAGNOSTICS WORKFLOW

DAY 4 - DASHBOARDS 101 (CREATE, WIDGETS/VIEWS, SAVE)

DAY 5 - WIDGET INTERACTIONS (MAKE DASHBOARDS DYNAMIC)

DAY 6 - DASHBOARD OPERATIONS (CLONE, EDIT, MANAGE, FAVORITES/RECENTS)

DAY 7 - CHECKPOINT #1 (SKILLS TEST + MINI INTERVIEW)

WEEK 2: DAYS 8-14 (Operations Depth: Alerts, Capacity, Compliance, Drift, Exec Views)

DAY 8 - ALERTS: WHAT MATTERS VS NOISE

DAY 9 - CAPACITY & PERFORMANCE STORY

DAY 10 - COMPLIANCE & CONFIGURATION DRIFT

DAY 11 - ROLE-BASED DASHBOARDS

DAY 12 - INCIDENT DAY #1 (TIMED)

DAY 13 - OPS STORYTELLING

DAY 14 - CHECKPOINT #2

WEEK 3: DAYS 15-21 (Logs Mastery: VCF Operations for Logs + Analyze + Dashboards/Alerts/Queries)

DAY 15 - WHAT LOGS ARE FOR (TROUBLESHOOTING VS AUDITING)

DAY 16 - DEPLOY VCF OPERATIONS FOR LOGS (OPERATIONS-LOGS)

DAY 17 - CONTENT PACKS & PREBUILT DASHBOARDS/ALERTS/QUERIES

DAY 18 - CREATE LOG DASHBOARDS (OPERATIONAL VIEWS)

DAY 19 - LOG-BASED ALERT PHILOSOPHY

DAY 20 - INCIDENT DAY #2 (TIMED)

DAY 21 - CHECKPOINT #3

WEEK 4: DAYS 22-30 (API + Automation + Capstone)

DAY 22 - VCF OPERATIONS API: SWAGGER & TOOLS

DAY 23 - AUTH PATTERNS (OPSTOKEN / SSO TOKEN)

Acquire token:

Use token (get alerts):

Release token:

DAY 24 - BUILD YOUR POSTMAN COLLECTION

DAY 25 - VCF SDK AWARENESS (PYTHON/JAVA) + SAMPLES

DAY 26 - VCF INSTALLER AUTOMATION LANDSCAPE

DAY 27 - VCF POWERCLI FUNDAMENTALS

==========

SCRIPT: Get-VcfOpsHealth.ps1

PURPOSE: Export cluster health summary to CSV

==========

Connect

Gather data

Calculate percentages

Export

Disconnect

==========

SCRIPT: Get-VcfOpsAlerts.ps1

PURPOSE: Pull alerts from VCF Operations API

==========

Ignore self-signed certs in lab

Acquire token

Get alerts

Display

Release token

DAY 28 - BUILD A "DAY-2 AUTOMATION" PACK

Configuration - replace with your values or use environment variables

DAY 29 - CAPSTONE: "OPS COMMAND CENTER"

DAY 30 - FINAL BOSS: MOCK INTERVIEW + LIVE DEMO

VCF OPERATIONS INTERVIEW PREP PACK - COMPLETE

Exact Answers | Scoring Rubrics | Whiteboard Prompts

A. YOUR 60-SECOND PITCH

B. COMPLETE QUESTION BANK WITH EXACT ANSWERS

DASHBOARDS & OPERATIONS (20 Questions)

API & AUTOMATION (20 Questions)

LOGS (20 Questions)

BEHAVIORAL / "TELL ME ABOUT A TIME" (10 Questions)

D. WHITEBOARD PROMPTS (WITH COMPLETE SOLUTIONS)

Whiteboard 1: "Design an Ops dashboard for VM performance triage"

Whiteboard 2: "Design a log dashboard for auth failures and suspicious activity"

Whiteboard 3: "Explain VCF Operations API auth and where Swagger lives"

E. MOCK INTERVIEW SCORING RUBRIC

VCF OPERATIONS LAB WORKBOOK - COMPLETE HANDS-ON GUIDE

Step-by-Step | Validation Checks | What Could Go Wrong

LAB TRACK A - DASHBOARDS (VCF Operations)

LAB A1: CREATE & CONFIGURE A DASHBOARD

LAB A2: MANAGE DASHBOARDS (CLONE, EDIT, FAVORITES)

LAB A3: CONFIGURE WIDGET INTERACTIONS

LAB TRACK B - API (VCF Operations)

LAB B1: FIND SWAGGER UI & EXPLORE ENDPOINTS

LAB B2: TOKEN AUTH WORKFLOW (POSTMAN)

LAB TRACK C - LOGS (VCF Operations for Logs + Analyze)

LAB C1: DEPLOY VCF OPERATIONS FOR LOGS

LAB C2: USE PREBUILT CONTENT PACKS

LAB TRACK D - AUTOMATION (OpenAPI / SDK / PowerCLI)

LAB D1: GENERATE A POSTMAN COLLECTION FROM OPENAPI

LAB D2: EXPLORE VCF SDK SAMPLES (PYTHON)

Suppress SSL warnings for lab (remove in production)

Read configuration from environment variables

Step 1: Acquire token

Step 2: Make an API call (list resources)

Step 3: Release token

LAB D3: INSTALL VCF POWERCLI & RUN A SMALL SCRIPT

VCF OPERATIONS 30-DAY MASTERY PLAN - EXTREMELY DETAILED

VCF 9.x Aligned | Every Action Spelled Out | Zero Assumptions

WEEKLY STRUCTURE (NON-NEGOTIABLE)

• Mon-Thu: New capability + hands-on lab (2-3 hours each day)

• Friday: "Incident Day" - timed troubleshooting simulation (2 hours)

• Saturday: Automation build day - scripts and API practice (3 hours)

• Sunday: Review + interview drill practice (2 hours)

WEEK 1: DAYS 1-7 (Foundation: Navigate, Inventory, Health, Dashboards Basics)

DAY 1 - VCF OPS MENTAL MODEL & UI MAP

Time required: 2-3 hours

OBJECTIVE: Be able to explain VCF constructs (fleet > instance > domains) and where Ops fits in the stack. Produce a 1-page navigation map.

STEP-BY-STEP ACTIONS:

1. Open your browser. Navigate to your VCF Operations UI:

• Type: `https://<your-ops-fqdn>/ui`

• Example: `https://vrops.lab.local/ui`

• Log in with your admin credentials (username: admin, password: your-password)

2. Once logged in, you land on the **Home** page. This is the VCF Operations **Launchpad**.

• At the top of the Launchpad you see three filter tabs:

• **All VMware Clouds** (shows everything across all cloud types)

• **VMware Cloud Foundation** (filters to VCF-managed resources)

• **vCenter** (filters to individual vCenter views)

• On the right side you see **Monitoring Accounts** (Private Clouds count, vCenters count, etc.)

• At the bottom you see **Appliances Health & Management** section

• You can pin up to 5 dashboards to this Product Home page

• Now look at the LEFT NAVIGATION panel. It has these sections:

INFRASTRUCTURE OPERATIONS (top section - direct sub-items, not expandable):

• **Diagnostic Findings** (correlated issue detection - 107+ signatures from Broadcom Support)

• **VCF Health** (full VCF stack health monitoring)

• **Dashboards & Reports** (custom/prebuilt dashboards + scheduled reports)

• **Alerts** (alert definitions, symptoms, recommendations, notifications)

• **Troubleshooting Workbench** (active troubleshooting sessions, relationship graphs)

• **Analyze** (log analysis - search, extract fields, saved queries; requires Operations for Logs)

• **Storage Operations** (vSAN + non-vSAN datastore monitoring, performance diagnostics)

• **Network Operations** (NSX instances, Edge clusters, transport nodes, VTEP/agent troubleshooting)

• **Data Protection & Recovery** (VMware Live Recovery integration)

• **Automation Central** (automate actions on schedules for maintenance windows)

• **Configurations** (policies, settings, apply policies to object groups)

EXPANDABLE SECTIONS (click the arrow to see sub-items):

• **Workload Operations** > (Business Applications, Telegraf monitoring, Service Discovery, vGPU, Platform Monitoring)

• **Fleet Management** > (Lifecycle Management, Identity & Access, Certificates, Passwords, Configuration Management, Tag Management)

• **Capacity** > (Cost Home, Configuring Cost, Capacity Optimization, What-If, Chargeback)

• **Security** > (Security Operations Dashboard, Compliance, Audit Events)

• **License Management** > (dynamic licensing, connected/disconnected mode)

• **Administration** > (User Management, Global Settings, Maintenance Schedules, Outbound Settings, Policies)

• **Developer Center** > (REST APIs, SDK documentation, programmatic access)

• Write each of these down on paper or in a document.

3. Understand the VCF Hierarchy (memorize this):

• **Fleet** = The top-level grouping. Your entire VCF deployment managed from VCF Operations.

• **Instance** = A specific VCF Operations instance (you can have multiple).

• **Workload Domain** = A logical grouping in VCF (Management Domain, VI Workload Domain).

• Inside each domain: **Clusters** > **Hosts (ESXi)** > **VMs**

• VCF Operations sits as the **unified management and operations control plane** for ALL of this.

• **IMPORTANT for VCF 9.0:** The SDDC Manager UI is deprecated. Its workflows (lifecycle, fleet, certs, passwords) have moved INTO VCF Operations and vSphere Client. VCF Operations is now THE single pane of glass.

4. Click through each left-nav section and note what sub-items appear:

• Click **Diagnostic Findings** > Note: "Active Findings" page, CSV export, 107+ known-issue signatures

• Click **VCF Health** > Note: VCF Instance Alerts, ESXi host issues, vCenter/vSAN/NSX health

• Click **Dashboards & Reports** > Note: Favorites, Recents, All categories; Manage tab for create/edit/clone

• Click **Alerts** > Note: Alert Definitions, Symptom Definitions, Recommendations, Notifications, Outbound Settings

• Click **Troubleshooting Workbench** > Note: search bar, active sessions, Object Relationship Graph/Tree

• Click **Analyze** > Note: Log search, saved queries, extracted fields, event types, trends

• Click **Storage Operations** > Note: vSAN dashboards, dedup/compression visibility, performance diagnostics

• Click **Network Operations** > Note: NSX Instances, Edge Clusters, Transport Nodes, VTEP State

• Expand **Fleet Management** > Note: Lifecycle, Identity & Access, Certificates, Passwords, Configuration Management, Tags

• Expand **Capacity** > Note: Cost Home, Capacity Optimization, What-If, Chargeback

• Expand **Security** > Note: Security Operations Dashboard, Compliance, Audit Events

• Expand **Administration** > Note: User Management, Global Settings, Policies, Outbound Settings

5. **CREATE YOUR DELIVERABLE - 1-Page Navigation Map:**

Open a new document (Word, Google Doc, or plain text). Create this exact structure:

```
VCF OPERATIONS NAVIGATION MAP (VCF 9.x - Verified from Broadcom TechDocs)
==========================================================================
LOGIN URL: https://<ops-fqdn>/ui

HOME / LAUNCHPAD:
  Tabs: [All VMware Clouds] [VMware Cloud Foundation] [vCenter]
  Right panel: Monitoring Accounts (Private Clouds, vCenters, Hosts, VMs)
  Bottom: Appliances Health & Management
  Feature: Pin up to 5 dashboards to Product Home

LEFT NAV STRUCTURE:

INFRASTRUCTURE OPERATIONS (direct items - no expand arrow):
■■■ Diagnostic Findings (107+ known-issue signatures, CSV export)
■■■ VCF Health (full stack: ESXi, vCenter, vSAN, NSX health)
■■■ Dashboards & Reports (Favorites/Recents/All + Manage + scheduled reports)
■■■ Alerts (definitions, symptoms, recommendations, notifications, outbound)
■■■ Troubleshooting Workbench (sessions, relationship graph/tree, run scripts)
■■■ Analyze (log search, saved queries, extracted fields, event types, trends)
■■■ Storage Operations (vSAN + non-vSAN datastores, dedup/compression, perf diag)
■■■ Network Operations (NSX instances, Edge clusters, transport nodes, VTEP state)
■■■ Data Protection & Recovery (VMware Live Recovery integration)
■■■ Automation Central (schedule automated actions for maintenance windows)
■■■ Configurations (policies, settings, apply to object groups)

WORKLOAD OPERATIONS > (expandable)
■■■ Business Applications (interconnected app/service health)
■■■ Product-Managed Telegraf (app service monitoring on VMs)
■■■ Open Source Telegraf (deploy Telegraf via cloud proxy)
■■■ Service Discovery (auto-discover services; network flow-based in 9.0)
■■■ vGPU Monitoring (VM GPU metrics from ESXi)
■■■ Platform Monitoring (Supervisor + VKS cluster auto-discovery)

FLEET MANAGEMENT > (expandable)
■■■ Lifecycle Management (bundles, prechecks, upgrades, combined ESX+NSX)
■■■ Identity & Access Management (SSO, OIDC, SAML, LDAP, SCIM)
■■■ Certificate Management (view, alert, auto-renew, VMCA/MSCA/OpenSSL CA)
■■■ Password Management (account status, expiry, fleet-wide remediation)
■■■ Configuration Management (drift detection, Git integration, PDF reports)
■■■ Tag Management (import/create/push tags to multiple vCenters, JSON export)

CAPACITY > (expandable)
■■■ Cost Home (centralized cost visualization)
■■■ Configuring Cost (SDDC costing, currency, comparisons)
■■■ Capacity Optimization (utilization, what-if exports, storage exclusion)
■■■ Chargeback (rate cards for supervisor architecture)
■■■ Showback / Billing (tenant/org cost transparency)

SECURITY > (expandable)
■■■ Security Operations Dashboard (security posture, user tracking)
■■■ Compliance (packs, benchmarks, rules, remediation)
■■■ Audit Events (cross-vCenter activity search, vSphere/vSAN/NSX events)
```

```
LICENSE MANAGEMENT > (expandable)
■■■ Dynamic licensing (connected + disconnected/air-gapped modes)

ADMINISTRATION > (expandable)
■■■ User Management / Access Control (roles, auth sources)
■■■ Global Settings (environment-wide config)
■■■ Maintenance Schedules (planned maintenance windows)
■■■ Outbound Settings (external notification channels)
■■■ Policies (operational policy management)
■■■ Metrics and Properties (metric/property definitions)

DEVELOPER CENTER > (expandable)
■■■ REST APIs, SDK docs, programmatic automation access

VCF HIERARCHY:
Fleet → Instance → Workload Domain → Cluster → Host → VM

WHERE OPS FITS (VCF 9.0 - CRITICAL CHANGE):
VCF Operations = THE unified management console (monitoring + lifecycle + fleet)
SDDC Manager UI = DEPRECATED in VCF 9.0 (workflows moved to VCF Operations)
vCenter / vSphere Client = compute management + some migrated SDDC Manager tasks
NSX Manager = network management plane

DEPLOYMENT MODELS:
Simple = single node + vSphere HA
High Availability = 3-node analytics cluster
Continuous Availability = dual fault domain
```

6. Save this document as `VCF_Ops_Navigation_Map.md` or `.docx`.

VALIDATION CHECK: Can you answer these without looking?

• Q: "What are the main left-nav sections in VCF Operations?" A: Infrastructure Operations (with 11 direct items), plus 7 expandable sections: Workload Operations, Fleet Management, Capacity, Security, License Management, Administration, Developer Center.

• Q: "Where do you go to create a new dashboard?" A: Infrastructure Operations > Dashboards & Reports > Manage

• Q: "What is the VCF hierarchy from top to bottom?" A: Fleet > Instance > Workload Domain > Cluster > Host > VM

• Q: "What happened to SDDC Manager in VCF 9.0?" A: The SDDC Manager UI is deprecated. Its workflows (lifecycle, fleet, certs, passwords, configuration management) moved into VCF Operations and vSphere Client. VCF Operations is now the single unified console.

• Q: "What are the three Launchpad tabs?" A: All VMware Clouds, VMware Cloud Foundation, vCenter

• Q: "Name 5 items under Infrastructure Operations." A: Diagnostic Findings, VCF Health, Dashboards & Reports, Alerts, Troubleshooting Workbench, Analyze, Storage Operations, Network Operations, Data Protection & Recovery, Automation Central, Configurations (any 5)

DAY 2 - INVENTORY & OBJECT MODEL

Time required: 2-3 hours

OBJECTIVE: Confidently trace object relationships (Cluster > Host > VM, etc.) using Inventory. Produce an object-relationship cheat sheet.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI: `https://<your-ops-fqdn>/ui`

2. Navigate to Inventory. In VCF 9.x, the inventory/environment view is accessible via:

• The **search bar** at the top of VCF Operations (type any object name to find it)

• The **Launchpad** Monitoring Accounts section (click through Private Clouds, vCenters, etc.)

• **Infrastructure Operations > Dashboards & Reports** (use predefined inventory dashboards)

• Or use the search/browse functionality within any section to navigate to objects

3. You will see a tree structure or object list. This is the object hierarchy. Expand it:

• Click the arrow next to **vSphere World**

• You will see your **vCenter Server** adapters listed

• Click the arrow next to a vCenter adapter

• You will see: **Datacenters**

• Expand a Datacenter: you see **Clusters**

• Expand a Cluster: you see **Hosts (ESXi)**

• Expand a Host: you see **VMs** running on that host

4. Click on any single object (for example, click on a Cluster name).

• The right panel shows **Summary**, **Alerts**, **Metrics**, **Properties**, **Relationships**

• Click the **Relationships** tab

• This shows you parent objects (what this cluster belongs to) and child objects (what is inside this cluster)

• Note: This is the critical skill. You must be able to trace UP and DOWN the tree.

5. Practice these specific traces (do each one):

• **Trace 1:** Pick any VM. Click on it. Go to Relationships tab. Find its Parent Host. Click on that Host. Find its Parent Cluster. Click on that Cluster. Find its Parent Datacenter. You just traced VM > Host > Cluster > Datacenter.

• **Trace 2:** Pick a Cluster. Go to Relationships. Find all Child Hosts. Count them. Find all VMs across those hosts.

• **Trace 3:** Pick a Datastore. Go to Relationships. Find which Hosts have access to it and which VMs reside on it.

6. Learn the object types VCF Operations tracks (write these down):

• **Compute:** vCenter, Datacenter, Cluster, Resource Pool, Host, VM

• **Storage:** Datastore, Datastore Cluster, vSAN

• **Network:** Distributed Switch, Port Group, NSX components

• **VCF-specific:** Workload Domain, SDDC Manager

7. **CREATE YOUR DELIVERABLE - Object Relationship Cheat Sheet:**

```
VCF OPERATIONS OBJECT RELATIONSHIP CHEAT SHEET
==============================================

COMPUTE PATH (most common trace):
vCenter Server
  ■■■ Datacenter
       ■■■ Cluster
            ■■■ ESXi Host
                 ■■■ Virtual Machine
                      ■■■ Virtual Disk / vNIC

STORAGE PATH:
Datastore Cluster
```

```
  ■■■ Datastore (VMFS / vSAN / NFS)
       ■■■ VM Files (.vmx, .vmdk)

NETWORK PATH:
Distributed Virtual Switch (DVS)
  ■■■ Distributed Port Group
       ■■■ VM vNIC

RELATIONSHIP TYPES:
- Parent: the object this object belongs to (Host's parent = Cluster)
- Child: objects contained within (Cluster's children = Hosts)
- Related: associated objects (Datastore relates to Hosts that mount it)

HOW TO TRACE IN THE UI:
1. Use the search bar at top of VCF Operations to find any object by name
2. Or navigate via Launchpad > Monitoring Accounts > drill into vCenters/clusters
3. Click any object to open its detail view
4. Click "Relationships" tab in right panel
5. Click parent/child links to navigate up/down the tree

KEY COUNTS TO KNOW FOR YOUR ENVIRONMENT:
- vCenter servers: ___
- Datacenters: ___
- Clusters: ___
- Hosts: ___
- VMs: ___
- Datastores: ___
```

VALIDATION CHECK:

• Q: "If I give you a VM name, how do you find what host it runs on?" A: Use the search bar at the top of VCF Operations, type the VM name, click the result, go to Relationships tab, find Parent Host.

• Q: "How do you find all VMs in a cluster?" A: Search for the cluster name, click it, go to Relationships tab, expand Child objects to see all hosts and their VMs.

DAY 3 - HEALTH & DIAGNOSTICS WORKFLOW

Time required: 2-3 hours

OBJECTIVE: Run a repeatable "first 15 minutes of triage" using health/diagnostic findings. Produce a triage runbook v1.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Go to **Home** page. Look at the summary cards:

• Note the overall health badge colors: Green (healthy), Yellow (warning), Orange (immediate attention), Red (critical)

• Write down the current state of each domain shown

3. Click **Alerts** under Infrastructure Operations in the left nav

• By default you see active alerts

• Click the **Severity** column header to sort by severity (Critical first)

• Note the filter options at the top:

• **Status:** Active, Cancelled

• **Criticality:** Critical, Immediate, Warning, Information

• **Time Range:** Last 24 hours, Last 7 days, etc.

• Set filter to: Status = Active, Criticality = Critical

• Write down how many critical alerts exist right now

4. For any critical alert, click on it to open the detail pane:

• Read the **Alert Name** (what is happening)

• Read the **Description** (detailed explanation)

• Read the **Recommendations** (VCF Ops tells you what to do)

• Note the **Object** (what is affected)

• Note the **Root Cause** section if present

• This is your triage starting point for any incident

5. Go to **Troubleshooting Workbench** under Infrastructure Operations in the left nav:

• You see a search bar, active sessions list, and recent searches

• Start a new troubleshooting session by searching for an object that has alerts

• The tool shows you an **Object Relationship Graph/Tree** with visual mapping of related issues

• You can group objects, search within the graph, and even run custom scripts on VMs

• Note: Troubleshooting sessions do NOT persist across logouts - complete your investigation in one session

6. Also check **Diagnostic Findings** under Infrastructure Operations:

• This is a single-pane-of-glass that correlates issues using 107+ known-issue signatures from Broadcom Support

• The **Active Findings** page shows current issues detected by automated scanning

• You can export findings to CSV for reporting

• This is different from Troubleshooting Workbench: Diagnostic Findings is automatic/proactive, Workbench is manual/reactive

7. Navigate to a specific Cluster (use search bar or browse from Launchpad):

• Click the **Summary** tab

• Look at the Health badge (colored circle)

• Look at the Workload badge

• Look at the Capacity remaining indicators

• These three together tell the story: Is it healthy? Is it busy? Does it have room?

7. **BUILD YOUR TRIAGE RUNBOOK v1:**

```
VCF OPERATIONS TRIAGE RUNBOOK - FIRST 15 MINUTES
==================================================

MINUTE 0-2: ASSESS SCOPE
Action: Go to Home page
Look at: Summary cards - which domains show non-green?
Question to answer: "Is this isolated to one object or widespread?"
What to write down: Number of critical/immediate alerts

MINUTE 2-5: IDENTIFY TOP ALERTS
Action: Go to Infrastructure Operations > Alerts
Filter: Status=Active, Severity=Critical, then Immediate
Sort by: Time (newest first)
For each critical alert, record:
  - Alert name
```

```
  - Affected object (name + type)
  - Time triggered
  - Recommendation text (copy/paste it)

MINUTE 5-8: TRACE THE AFFECTED OBJECT
Action: Click the affected object in the alert
Go to: Relationships tab
Question: "What else depends on this object?"
  - If it is a Host: What VMs run on it? What cluster is it in?
  - If it is a Datastore: What VMs use it? What hosts mount it?
  - If it is a VM: What host is it on? Is the host healthy?

MINUTE 8-12: CHECK METRICS
Action: On the affected object, click Metrics tab
Look at: CPU usage, Memory usage, Disk latency, Network throughput
Compare to: Normal baseline (is this a spike or sustained?)
Time range: Set to Last 24 hours, then Last 7 days for trend

MINUTE 12-15: DOCUMENT & ESCALATE
Write in your incident notes:
  - What: [alert name]
  - Where: [object name and path]
  - Impact: [what is affected downstream - VMs, users]
  - Probable cause: [from Recommendations + your trace]
  - Next step: [fix it yourself / escalate / monitor]

ESCALATION CRITERIA:
- Critical alert on Management Domain cluster/host = ESCALATE IMMEDIATELY
- Critical alert affecting >10 VMs = ESCALATE
- Capacity below 10% on any cluster = ESCALATE
- Alert present >1 hour with no auto-resolution = INVESTIGATE
```

VALIDATION CHECK:

• Q: "Walk me through your first 15 minutes when you get paged." Use the runbook above verbatim.

• Q: "How do you determine blast radius?" A: "I click the affected object, go to Relationships, and trace what depends on it - child VMs, connected datastores, dependent services."

DAY 4 - DASHBOARDS 101 (CREATE, WIDGETS/VIEWS, SAVE)

Time required: 2-3 hours

OBJECTIVE: Create a dashboard from scratch, add widgets, save it. Produce "My First Ops Dashboard" with screenshot evidence.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Click **Dashboards & Reports** under Infrastructure Operations in the left nav

3. Click the **Manage** tab, then click the **Create** button (usually a "+" icon or "Create Dashboard" button)

4. A new blank dashboard canvas opens. You must configure:

Step 4a - Name your dashboard:

• In the dashboard name field at the top, type exactly: `Daily Health Overview - [Your Name]`

• Example: `Daily Health Overview - Michael Hayes`

Step 4b - Add your first widget (Health Status):

• On the left side, you see a widget panel/palette

• Find the widget type: **Scoreboard** (or **Heat Map** depending on what you want)

• Drag it onto the dashboard canvas

• A configuration dialog appears:

• **Title:** Type `Cluster Health Status`

• **Object Type:** Select `Cluster Compute Resource`

• **Metric:** Select `Badge | Health`

• Click **Save** or **Apply**

Step 4c - Add a second widget (Top VMs by CPU):

• Find widget type: **Top-N** (or **Top Alerts**)

• Drag it onto the canvas next to the first widget

• Configuration:

• **Title:** Type `Top 10 VMs by CPU Usage`

• **Object Type:** Select `Virtual Machine`

• **Metric:** Select `CPU | Usage (%)`

• **Count:** Set to `10`

• Click **Save** or **Apply**

Step 4d - Add a third widget (Alert List):

• Find widget type: **Alert List**

• Drag it onto the canvas below the first two

• Configuration:

• **Title:** Type `Active Critical Alerts`

• **Filter:** Set severity to `Critical` and `Immediate`

• Click **Save** or **Apply**

Step 4e - Add a fourth widget (Capacity Overview):

• Find widget type: **Capacity Overview** or **Scoreboard**

• Drag it onto the canvas

• Configuration:

• **Title:** Type `Cluster Capacity Remaining`

• **Object Type:** Select `Cluster Compute Resource`

• **Metric:** Select `Badge | Capacity Remaining`

• Click **Save** or **Apply**

5. **Arrange the widgets:**

• Drag widgets to arrange them in a 2x2 grid:

• Top-left: Cluster Health Status

• Top-right: Top 10 VMs by CPU

• Bottom-left: Active Critical Alerts

• Bottom-right: Cluster Capacity Remaining

6. Click **Save** (disk icon or Save button at top of dashboard editor)

7. **Take a screenshot:**

• Press `Windows + Shift + S` (Windows Snipping Tool)

• Select the entire dashboard

• Save the screenshot as `My_First_Ops_Dashboard.png`

DELIVERABLE: Your saved dashboard named "Daily Health Overview - [Your Name]" + the screenshot file.

VALIDATION CHECK:

• Q: "How do you create a dashboard in VCF Operations?" A: "I go to Infrastructure Operations, then Dashboards & Reports, click the Manage tab, click Create, name the dashboard, drag widgets from the palette onto the canvas, configure each widget with an object type and metric, arrange them, and click Save."

• Q: "What widgets would you put on a daily health dashboard?" A: "A health scoreboard for clusters, a Top-N for busiest VMs by CPU, an active critical alerts list, and a capacity remaining scoreboard."

DAY 5 - WIDGET INTERACTIONS (MAKE DASHBOARDS DYNAMIC)

Time required: 2-3 hours

OBJECTIVE: Configure widget interactions so clicking one widget updates another. Produce a dashboard that filters dynamically.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Go to **Infrastructure Operations > Dashboards & Reports** > find your "Daily Health Overview" dashboard > click **Edit** (pencil icon)

3. **Understand widget interactions:**

• An interaction means: "When I click an object in Widget A, Widget B updates to show data for that object"

• Example: Click a cluster in the Health Scoreboard > the Top VMs widget updates to show only VMs in that cluster

• This is what makes dashboards dynamic instead of static

4. **Configure your first interaction:**

• Click on the **Cluster Health Status** widget (the scoreboard you made on Day 4)

• Look for an **Edit Interactions** option (pencil icon on the widget, or right-click > Edit Interactions, or look in the widget's three-dot menu)

• In the interaction editor:

• **Source widget:** Cluster Health Status (this is the one you are editing)

• **Target widget:** Select `Top 10 VMs by CPU Usage`

• **Interaction type:** Select `Selected Object` (meaning: pass the selected cluster to the target widget)

• Click **Apply** or **Save**

5. **Configure a second interaction:**

• Same source widget: Cluster Health Status

• **Target widget:** Select `Active Critical Alerts`

• **Interaction type:** `Selected Object`

• Click **Apply**

6. **Test the interactions:**

• Save the dashboard

• Click on a specific cluster in the Health Scoreboard widget

• Verify: the Top VMs widget now shows only VMs from that cluster

• Verify: the Alert List widget now shows only alerts for that cluster

• If it works, your dashboard is now dynamic

7. **Add a navigation element (optional but impressive):**

• Edit the dashboard again

• Add a new widget: **Object List** or **Inventory Tree**

• Configure it to show Clusters

• Set interactions from this widget to all other widgets

• Now this acts as your "selector" - click any cluster and the whole dashboard updates

8. Save the dashboard.

DELIVERABLE: Your "Daily Health Overview" dashboard now updates dynamically when you click a cluster. Screenshot the before/after (click a cluster, show how other widgets change).

VALIDATION CHECK:

• Q: "What are widget interactions?" A: "Widget interactions let you connect widgets so that selecting an object in one widget automatically filters or updates other widgets on the same dashboard. For example, clicking a cluster in a health scoreboard can filter an alert list to show only alerts for that cluster."

• Q: "Why are interactions important?" A: "They turn a static dashboard into a dynamic investigation tool. Instead of building 20 separate dashboards, I build one with interactions so I can drill into any object on the fly."

DAY 6 - DASHBOARD OPERATIONS (CLONE, EDIT, MANAGE, FAVORITES/RECENTS)

Time required: 2 hours

OBJECTIVE: Manage dashboards at scale. Clone, edit, organize with favorites/recents. Produce a "Daily Driver" dashboard pinned.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Go to **Infrastructure Operations > Dashboards & Reports**

• You will see dashboard categories: **Favorites**, **Recents**, and **All**

3. **Clone a prebuilt dashboard:**

• Browse through the dashboards under **All**. Find a prebuilt/predefined dashboard (they ship with VCF Operations). Look for one named something like "vSphere VM Performance" or "Cluster Capacity"

• Click on the dashboard to open it

• Click the **three-dot menu** (or "Actions" menu) at the top right of the dashboard

• Select **Clone**

• A new dashboard is created with the name "[Original Name] - Clone"

• Click **Edit** on the cloned dashboard

• Rename it to: `NOC - VM Performance Overview`

• Modify widgets as needed (change time ranges, add filters)

• Click **Save**

4. **Edit an existing dashboard:**

• Find your "Daily Health Overview" dashboard

• Click **Edit** (pencil icon)

• Add one more widget: a **Metric Chart** widget

• Title: `CPU Trend - Last 7 Days`

• Object Type: Cluster Compute Resource

• Metric: CPU | Usage (%)

• Time Range: Last 7 Days

• Save

5. **Use Favorites:**

• Go to **Infrastructure Operations > Dashboards & Reports**

• Find your "Daily Health Overview" dashboard

• Click the **star icon** next to it (or right-click > Add to Favorites)

• The star turns solid/filled = it is now a Favorite

• Verify: Click **Favorites** filter at the top of the dashboard list. Your dashboard should appear.

6. **Use Recents:**

• Open 3-4 different dashboards (just click into them, then back out)

• Look for a **Recents** filter or section in the Dashboard area

• Verify that the dashboards you just viewed appear in Recents

7. **Manage Dashboards (permissions, sharing):**

• Go to **Infrastructure Operations > Dashboards & Reports > Manage**

• Find your "Daily Health Overview" dashboard

• Click it to see options:

• **Share:** Set to "Share with everyone" or specific user groups

• **Delete:** Option to delete dashboards you no longer need

• **Export:** Export dashboard as a file for backup or migration

8. **Pin your "Daily Driver":**

• Your "Daily Health Overview" should now be:

• Favorited (starred)

• Set as your landing dashboard (if the option exists: Dashboard > Set as Default)

• This is your "Daily Driver" - the first thing you see when you log in

DELIVERABLE: "Daily Driver" dashboard is favorited, shows in Favorites list, and you have a cloned "NOC" dashboard.

VALIDATION CHECK:

• Q: "How do you avoid dashboard sprawl?" A: "I use the Favorites category in Dashboards & Reports to pin the dashboards I actually use daily, I clone prebuilt dashboards instead of building from scratch, I review and delete unused dashboards monthly through the Manage tab, and I name dashboards with a prefix convention like NOC-, EXEC-, or OPS- so they are easy to find."

• Q: "How do you share a dashboard with another team?" A: "I go to Infrastructure Operations > Dashboards & Reports > Manage, select the dashboard, and use the Share option to grant access to specific user groups."

DAY 7 - CHECKPOINT #1 (SKILLS TEST + MINI INTERVIEW)

Time required: 2-3 hours

WHAT YOU MUST DO:

Part 1 - Skills Demo (45 minutes):

1. Without any notes, do the following in the VCF Operations UI:

• Create a new dashboard from scratch

• Name it: `Checkpoint 1 - [Your Name]`

• Add at least 4 widgets (Health, Top-N, Alerts, Capacity)

• Configure at least 2 widget interactions

• Save it

• Time yourself. Target: under 15 minutes.

2. Navigate the object model:

• Pick any VM, trace it to its Host, then Cluster, then Datacenter

• Do it in under 2 minutes

3. Run a triage:

• Go to Alerts, find the highest severity alert

• Trace the affected object

• Explain (out loud or written) what the impact is and what you would do

Part 2 - Mini Interview (30 minutes):

Answer these questions out loud (record yourself or have someone ask you):

Q1: "What is VCF Operations and where does it fit in the VCF stack?"

EXACT ANSWER: "VCF Operations is the unified management console in VMware Cloud Foundation 9. In VCF 9.0, the SDDC Manager UI was deprecated, and its workflows for lifecycle management, fleet management, certificates, passwords, and configuration management all moved into VCF Operations. So it is now the single pane of glass for everything: health monitoring, performance analytics, capacity planning, compliance, dashboards, log analysis, and lifecycle operations. The left nav has Infrastructure Operations with items like Diagnostic Findings, VCF Health, Dashboards & Reports, Alerts, Troubleshooting Workbench, Analyze for logs, Storage Operations, and Network Operations. Then expandable sections for Workload Operations, Fleet Management, Capacity, Security, License Management, Administration, and Developer Center. The object hierarchy goes Fleet, Instance, Workload Domain, Cluster, Host, VM."

Q2: "Walk me through a dashboard you built."

EXACT ANSWER: "I built a Daily Health Overview dashboard for our operations team. It has four widgets: a cluster health scoreboard that shows green/yellow/red status for each cluster, a Top-10 VMs by CPU chart to spot

resource hogs, an active critical alerts list filtered to only Critical and Immediate severity, and a cluster capacity remaining scoreboard. The key feature is widget interactions - when I click a specific cluster in the health scoreboard, the VM list and alert list automatically filter to show only data for that cluster. This lets me go from 'something is wrong in cluster X' to 'here are the specific VMs and alerts' in one click."

Q3: "How do you triage an issue in VCF Operations?"

EXACT ANSWER: "I follow a 15-minute triage runbook. Minutes 0-2: I check VCF Health under Infrastructure Operations and the Launchpad summary to assess scope - is this one object or widespread? I also check Diagnostic Findings which automatically correlates issues against 107 known-issue signatures from Broadcom Support. Minutes 2-5: I go to Infrastructure Operations > Alerts, filter to Active Critical, sort by newest, and record the alert name, affected object, and recommendations for each. Minutes 5-8: I click into the affected object, go to Relationships to understand blast radius - what depends on this object? If a host is down, I need to know which VMs are on it. Minutes 8-12: I check the Metrics tab to see if this is a spike or sustained issue, looking at CPU, memory, disk latency, and network. If log evidence is needed, I use Infrastructure Operations > Analyze to search logs. Minutes 12-15: I document findings and either fix it or escalate based on severity and impact."

Q4: "What are widget interactions and why do they matter?"

EXACT ANSWER: "Widget interactions connect widgets on a dashboard so selecting an object in one widget filters or updates other widgets. For example, clicking a cluster in a health scoreboard automatically shows only that cluster's VMs in the Top-N widget and only that cluster's alerts in the alert list. They matter because they turn static wallpaper dashboards into dynamic investigation tools. Instead of maintaining 20 single-purpose dashboards, I build one interactive dashboard that lets me drill down on the fly during triage."

SCORING (be honest with yourself):

• Created dashboard with 4+ widgets in under 15 min? YES/NO

• Configured working interactions? YES/NO

• Traced VM > Host > Cluster in under 2 min? YES/NO

• Answered all 4 questions without hesitation? YES/NO

• Target: 4/4 YES to proceed to Week 2

WEEK 2: DAYS 8-14 (Operations Depth: Alerts, Capacity, Compliance, Drift, Exec Views)

DAY 8 - ALERTS: WHAT MATTERS VS NOISE

Time required: 2-3 hours

OBJECTIVE: Define alert hygiene. Build an alert taxonomy. Know what to action vs ignore.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Go to **Infrastructure Operations > Alerts** in the left nav

3. **Audit your current alert landscape:**

• Set filter: Status = Active, Time Range = Last 7 days

• Write down the total count of alerts by severity:

• Critical: ___

• Immediate: ___

• Warning: ___

• Information: ___

• If Critical + Immediate > 50, you have alert noise problems

4. **Understand alert types in VCF Operations:**

• Within the **Alerts** section, navigate to **Alert Definitions**

• In VCF 9.x, alerts support: simple alerts, standard alerts, and log-based alerts

• You also have **Symptom Definitions**, **Recommendations**, and **Alert Groups**

• Browse through the definitions. Notice categories:

• **Health alerts:** Object health degraded (CPU contention, memory pressure)

• **Capacity alerts:** Running out of resources

• **Compliance alerts:** Configuration drift detected

• **Availability alerts:** Object down or unreachable

• **Performance alerts:** Latency, throughput degradation

5. **Build your alert taxonomy (P1-P4):**

• Open a document and create this exact classification:

```
ALERT TAXONOMY TEMPLATE
========================

P1 - CRITICAL (respond in <15 minutes)
Definition: Service-impacting, production down, data at risk
Examples:
  - Host not responding
  - Datastore out of space (<5% remaining)
  - vCenter service down
  - Management domain cluster health = Red
Action: Page on-call, begin triage immediately, update status page
Owner: On-call engineer
MTTR target: <1 hour

P2 - IMMEDIATE (respond in <1 hour)
Definition: Significant degradation, not yet service-impacting
Examples:
  - Cluster CPU usage >90% sustained 30 min
  - Memory contention on >3 hosts in cluster
  - Certificate expiring in <7 days
  - Capacity remaining <20% on any cluster
Action: Assign to team lead, begin investigation, schedule fix
Owner: Ops team lead
MTTR target: <4 hours

P3 - WARNING (respond within business day)
Definition: Non-urgent, could become P2 if ignored
Examples:
  - Single VM high CPU (not business-critical)
  - Capacity remaining <40% (trending toward limit)
  - Configuration drift detected on non-production cluster
  - NTP skew >1 second
Action: Add to daily standup queue, investigate during business hours
Owner: Assigned ops engineer
MTTR target: <24 hours

P4 - INFORMATIONAL (review weekly)
```

```
Definition: FYI, no action required unless pattern emerges
Examples:
  - VM powered off/on
  - Snapshot older than 7 days
  - Informational health status changes
Action: Review in weekly ops meeting, batch-resolve
Owner: Anyone
MTTR target: N/A (review cadence)
```

6. **Reduce noise - identify the top offenders:**

• Back in Alert List, sort by **Count** or look for alerts with the same name appearing many times

• These are your "noisy" alerts

• For each noisy alert, ask: "Does this require human action?" If no, consider:

• Raising the threshold (e.g., change CPU alert from 80% to 90%)

• Reducing the trigger sensitivity (e.g., must sustain for 10 min instead of 5 min)

• Disabling the alert definition entirely if it is never actionable

7. **Set up alert notifications (know how, even if you do not configure in lab):**

• Within the **Alerts** section, navigate to **Notifications** and **Outbound Settings**

• **Outbound Settings** configures WHERE notifications go (email server, SNMP trap, webhook endpoint, REST API)

• **Notifications** configures WHICH alerts trigger notifications and to WHOM

• **Payload Templates** let you customize the notification format

• Best practice: P1 alerts = email + page, P2 = email, P3-P4 = dashboard only

DELIVERABLE: Your completed P1-P4 Alert Taxonomy Template document.

VALIDATION CHECK:

• Q: "How do you handle alert noise?" A: "I start by auditing the alert landscape - counting active alerts by severity over the last 7 days. If Critical plus Immediate exceeds 50, there is a noise problem. I sort by frequency to find the top offenders - alerts that fire repeatedly. For each noisy alert, I ask whether it requires human action. If not, I raise the threshold, extend the trigger duration, or disable it. I classify all alerts into P1 through P4 categories with clear ownership and MTTR targets. P1 pages the on-call engineer, P2 goes to the team lead, P3 goes into the daily standup queue, and P4 gets reviewed weekly."

DAY 9 - CAPACITY & PERFORMANCE STORY

Time required: 2-3 hours

OBJECTIVE: Explain "capacity risk vs performance risk" and build a weekly capacity report outline.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Expand **Capacity** in the left nav (click the arrow to see sub-items: Cost Home, Configuring Cost, Capacity Optimization, Chargeback, Showback/Billing).

3. **Understand the two risks:**

• **Capacity risk** = "We are running out of room." (Time to exhaustion, remaining capacity in CPU/Memory/Storage)

• **Performance risk** = "We are running slowly right now." (Current contention, latency, queue depth)

• Capacity risk is forward-looking. Performance risk is current-state.

• Example: A cluster at 70% CPU is not a performance risk today, but if it is growing 5% per week, it is a capacity risk in 6 weeks.

4. Click into **Capacity Optimization** (under the Capacity section):

• Select a Cluster

• Look at:

• **Time Remaining:** How many days until CPU/Memory/Storage is exhausted at current growth rate

• **Capacity Remaining (%):** Current remaining percentage

• **Reclaimable Capacity:** Oversized VMs, idle VMs, snapshots that can be cleaned up

5. Within **Capacity Optimization**, look for **What-If Scenarios** (VCF 9.x supports exporting What-If results):

• This lets you model: "What if I add 50 VMs to this cluster?"

• Or: "What if I remove 2 hosts for maintenance?"

• Or: "What if I migrate 100 VMs from cluster A to cluster B?"

• Practice creating one What-If scenario to understand the tool.

6. Navigate to a specific Cluster (use search bar or browse via Launchpad) > **Metrics** tab:

• Look at current performance metrics:

• CPU Usage (%) - is it spiking above 80%?

• Memory Usage (%) - is it sustained above 85%?

• Disk Latency (ms) - is read/write latency above 20ms?

• Network Throughput - any saturation?

• These are PERFORMANCE indicators (current state, not future projection)

7. **BUILD YOUR DELIVERABLE - Weekly Capacity Report Outline:**

```
WEEKLY CAPACITY REPORT - [DATE RANGE]
=====================================

EXECUTIVE SUMMARY:
[1-2 sentences: Are we healthy? Any clusters at risk? Key action needed?]
Example: "All production clusters are healthy with >30% capacity remaining.
Dev cluster 2 will exhaust CPU in approximately 45 days at current growth
rate. Recommend right-sizing 12 oversized VMs to reclaim capacity."

SECTION 1: CAPACITY STATUS BY CLUSTER
| Cluster Name | CPU Remaining % | Memory Remaining % | Storage Remaining % | Days to Exhaustion (CPU) |
|---|---|---|---|---|
| Prod-Cluster-01 | 45% | 52% | 61% | 120 days |
| Prod-Cluster-02 | 38% | 41% | 55% | 90 days |
| Dev-Cluster-01 | 22% | 30% | 40% | 45 days |

SECTION 2: TOP CAPACITY RISKS
1. [Cluster name] - [Which resource] - [Days remaining] - [Recommended action]
2. ...

SECTION 3: RECLAIMABLE CAPACITY
- Oversized VMs (CPU allocated > used): [count] VMs, [amount] reclaimable
- Idle VMs (powered on, no activity >7 days): [count] VMs
- Old snapshots (>7 days): [count] snapshots, [size] GB

SECTION 4: PERFORMANCE HIGHLIGHTS (LAST 7 DAYS)
- Peak CPU usage event: [cluster] hit [%] on [date/time]
```

```
- Peak memory usage event: [cluster] hit [%] on [date/time]
- Any disk latency spikes: [yes/no, details]

SECTION 5: RECOMMENDATIONS
1. [Right-size these VMs]
2. [Delete these snapshots]
3. [Plan hardware procurement for cluster X by date Y]

NEXT REVIEW: [Date]
```

VALIDATION CHECK:

• Q: "What is the difference between capacity risk and performance risk?" A: "Capacity risk is forward-looking - it answers 'when will we run out of resources?' based on growth trends and remaining headroom. Performance risk is current-state - it answers 'are users experiencing slowness right now?' based on real-time contention metrics like CPU ready time, memory ballooning, and disk latency. A cluster can have no performance risk today but high capacity risk because it is growing fast."

• Q: "What would you report weekly?" A: "I present a capacity report that covers five sections: an executive summary with the headline, capacity status by cluster showing remaining percentages and days to exhaustion, top capacity risks ranked by urgency, reclaimable capacity from oversized VMs, idle VMs, and old snapshots, and specific recommendations with actions and dates."

DAY 10 - COMPLIANCE & CONFIGURATION DRIFT

Time required: 2-3 hours

OBJECTIVE: Talk through drift, why it matters, and what should be monitored. Build a drift watchlist.

STEP-BY-STEP ACTIONS:

1. Log into VCF Operations UI.

2. Expand **Security** in the left nav. You will see three sub-items:

• **Security Operations Dashboard** (overall security posture)

• **Compliance** (benchmark checking)

• **Audit Events** (who did what, when)

3. **Understand what compliance and drift mean in VCF 9.x:**

• **Compliance** (under Security) = checking configurations against known-good baselines or regulatory standards

• VCF Operations includes compliance packs with upgraded and new frameworks

• **Configuration drift** (under Fleet Management > Configuration Management) = scheduled drift detection for vCenter and cluster objects, with Git integration for template versioning and PDF drift reports

• These are TWO different features in two different places: Security > Compliance checks against benchmarks; Fleet Management > Configuration Management checks for drift from your own baselines

4. Click **Security > Compliance:**

• Browse available compliance packs and benchmarks:

• vSphere Security Configuration Guide

• CIS Benchmarks for ESXi

• Custom benchmarks (if defined)

• Click into one to see what rules it checks (examples):

• SSH disabled on ESXi hosts
• NTP configured correctly
• Lockdown mode enabled
• SNMP community strings changed from default
• Syslog configured to forward logs
• Firewall rules match baseline
• You can discover non-compliant objects and get remediation guidance

5. Now check **Fleet Management > Configuration Management:**

• This is where you do scheduled drift detection against YOUR baselines (not just industry benchmarks)
• It supports Git integration for versioning your configuration templates
• You can download PDF drift reports
• You can track VCP drift status
• Active and historical job tracking is available

6. Also check **Security > Audit Events:**

• This provides cross-vCenter activity searching
• Examines events across vSphere, vSAN, and NSX
• Default 24-hour view, but customizable
• This is your "who did what" investigation tool

6. **BUILD YOUR DELIVERABLE - Drift Watchlist (Top 10 Settings to Track):**

```
CONFIGURATION DRIFT WATCHLIST
==============================

These are the 10 settings I monitor for unauthorized changes.
Any change to these without a change ticket = investigate immediately.

1. ESXi SSH Service Status
   Baseline: DISABLED on all hosts
   Why: SSH enabled = potential attack vector + audit finding
   Check: Security > Compliance or Fleet Management > Configuration Management

2. ESXi NTP Configuration
   Baseline: NTP configured to [your NTP servers], service running
   Why: Time skew breaks logs correlation, certificate validation, vSAN
   Check: Security > Compliance or esxcli system ntp get

3. ESXi Syslog Forwarding
   Baseline: Forwarding to [your syslog server/Operations-Logs]
   Why: Without log forwarding, you lose forensic evidence
   Check: esxcli system syslog config get

4. ESXi Lockdown Mode
   Baseline: NORMAL lockdown mode enabled
   Why: Prevents direct host access bypassing vCenter
   Check: Security > Compliance

5. vCenter SSO Password Policy
   Baseline: Minimum 12 chars, complexity enabled, lockout after 5 attempts
   Why: Weak passwords = brute force risk
   Check: vCenter Administration > SSO > Configuration

6. ESXi Firewall Rules
   Baseline: Only required services allowed (no wildcard "all" rules)
   Why: Open firewalls = lateral movement risk
```

```
   Check: esxcli network firewall ruleset list

7. VM Hardware Version
   Baseline: Minimum version 19 (vSphere 7+) or 21 (vSphere 8+)
   Why: Old hardware versions miss security features
   Check: VM properties or Security > Compliance

8. Datastore Access Permissions
   Baseline: Only authorized hosts mount each datastore
   Why: Unauthorized access = data exposure risk
   Check: Datastore > Hosts tab

9. Distributed Switch Configuration
   Baseline: Promiscuous mode DISABLED, MAC changes DISABLED, Forged transmits DISABLED
   Why: These settings enabled = network sniffing possible
   Check: DVS > Security settings

10. Certificate Validity
    Baseline: All certs valid, expiry >30 days
    Why: Expired certs = service outages + security warnings
    Check: VCF Operations Alerts or SDDC Manager cert dashboard
```

VALIDATION CHECK:

• Q: "What is configuration drift and why does it matter?" A: "Configuration drift is when a system's actual configuration deviates from its approved baseline without going through change management. It matters for three reasons. First, security: an enabled SSH service or a disabled firewall rule is an attack surface. Second, stability: NTP skew can break vSAN, log correlation, and certificate validation. Third, compliance: regulatory audits like SOX, HIPAA, or PCI require proof that configurations match approved baselines. In VCF Operations, I use the Compliance feature to check against hardening guides and CIS benchmarks, and I maintain a top-10 drift watchlist of the settings that cause the most damage when they change."

• Q: "What are your top 5 drift settings?" Pick any 5 from the list and explain each one.

DAY 11 - ROLE-BASED DASHBOARDS

Time required: 3 hours

OBJECTIVE: Build one dashboard per audience (NOC / VMware Ops / Exec). Produce 3 dashboard wireframes.

STEP-BY-STEP ACTIONS:

1. **Understand the three audiences:**

NOC (Network Operations Center):

• Needs: Real-time status, active alerts, service up/down, quick triage entry points

• Does NOT need: Capacity trends, compliance details, exec summaries

• Refresh: Every 30-60 seconds (auto-refresh)

• Tone: "What is broken RIGHT NOW?"

VMware Ops (your team):

• Needs: Health trends, capacity planning, performance hotspots, drift/compliance, recent changes

• Refresh: Every 5 minutes

• Tone: "What needs attention TODAY and what is trending for THIS WEEK?"

Executive:

• Needs: Green/yellow/red status per domain, capacity runway in weeks/months, cost/efficiency metrics, risk summary

• Does NOT need: Individual VM details, raw alert lists, technical metrics

• Refresh: On-demand (weekly review)

• Tone: "Are we healthy? Do we need to buy anything? What risks exist?"

2. **Build wireframe 1: NOC Dashboard**

• In VCF Operations, create a new dashboard: `NOC - Real-Time Status`

• Widgets (left to right, top to bottom):

• **Row 1:** Health Heat Map (all clusters, color-coded) | Active Alert Count by Severity (scoreboard: Critical=red number, Immediate=orange)

• **Row 2:** Alert List (filtered: Active, Critical + Immediate only, sorted newest first) — full width

• **Row 3:** Host Status List (show all hosts, health badge) | Top 5 VMs by CPU Usage

• Interactions: Click a cluster in Heat Map > filters Alert List and Host Status to that cluster

• Auto-refresh: Set to 60 seconds

3. **Build wireframe 2: VMware Ops Dashboard**

• Create: `OPS - Daily Operations View`

• Widgets:

• **Row 1:** Cluster Health Scoreboard | Cluster Capacity Remaining Scoreboard

• **Row 2:** CPU Trend (7 day line chart) | Memory Trend (7 day line chart)

• **Row 3:** Active Alerts (all severities, last 24h) | Compliance Score Summary

• **Row 4:** Top 10 Oversized VMs | Recent Configuration Changes

• Interactions: Click cluster > updates all widgets

4. **Build wireframe 3: Executive Dashboard**

• Create: `EXEC - Weekly Infrastructure Summary`

• Widgets:

• **Row 1:** Overall Health Score (single large green/yellow/red badge) | Capacity Runway (text widget: "X weeks until CPU exhaustion")

• **Row 2:** Health by Workload Domain (scoreboard: Management=Green, Prod=Green, Dev=Yellow)

• **Row 3:** Risk Summary (text widget listing top 3 risks) | Efficiency Score (% of resources actually used vs allocated)

• No interactions needed — this is a "glance and go" dashboard

• Minimal detail, maximum clarity

5. Document each wireframe (draw on paper or in a doc):

• Dashboard name

• Target audience

• Widget layout (grid sketch)

• Interactions defined

• Refresh rate

• What question this dashboard answers

DELIVERABLE: 3 dashboard wireframes (NOC, Ops, Exec) either built in the UI or documented as design specs.

VALIDATION CHECK:

• Q: "How do you decide what goes on a NOC dashboard vs an exec dashboard?" A: "The NOC dashboard answers 'what is broken right now' - it needs real-time data, active critical alerts, service status, and auto-refresh every 60 seconds. The NOC operator needs to see the problem and start triage in seconds. The exec dashboard answers 'are we healthy and do we need to spend money' - it shows aggregate health by domain, capacity runway in weeks, and a risk summary. No individual VM details, no raw alert lists. One is operational and urgent, the other is strategic and periodic."

DAY 12 - INCIDENT DAY #1 (TIMED)

Time required: 2 hours

OBJECTIVE: Simulate a real incident, triage it under time pressure, document findings.

SIMULATION SCENARIO: "Multiple users report application slowness at 9:15 AM."

SET A TIMER FOR 30 MINUTES. START NOW.

STEP-BY-STEP TRIAGE:

Minutes 0-2: Scope Assessment

1. Open VCF Operations > Home page
2. Look at summary cards. Note which domains/clusters show degraded health.
3. Write down: "Scope: [widespread/isolated], Domains affected: [list]"

Minutes 2-5: Alert Investigation

4. Go to Alerts > Alert List
5. Filter: Active, Critical + Immediate, Last 1 hour
6. Sort by newest first
7. For each alert, write down:

• Alert name: ___

• Object affected: ___

• Triggered at: ___

• Recommendation: ___

Minutes 5-10: Object Investigation

8. Click the most critical affected object
9. Go to Relationships tab - what depends on it?
10. Go to Metrics tab - what metric is abnormal?
11. Write down: "Root object: ___, Impact: ___ VMs / ___ users estimated"

Minutes 10-15: Performance Check

12. Check the affected cluster's CPU, Memory, Disk Latency metrics
13. Look at the time range for the last 2 hours - when did the anomaly start?
14. Write down: "Anomaly started at: ___, Metric: ___, Value: ___"

Minutes 15-20: Root Cause Hypothesis

15. Based on what you found, write a hypothesis:

"I believe the slowness is caused by [specific finding] on [specific object], which affects [N VMs / services]."

Minutes 20-25: Remediation Plan

16. Write down what you would do to fix it:

• Immediate: [e.g., vMotion VMs off the affected host, restart a service]

• Short-term: [e.g., add capacity, right-size VMs]

• Long-term: [e.g., set up alert for this condition, automate the response]

Minutes 25-30: Documentation

17. Write your post-incident notes:

```
INCIDENT NOTES - [DATE]
========================
Reported: 9:15 AM - Users report application slowness
Triage started: [time]
Triage completed: [time]

FINDINGS:
- Alert(s): [list]
- Affected object(s): [list]
- Root cause hypothesis: [statement]
- Impact: [N VMs, N users, N services]

REMEDIATION:
- Immediate: [what you did/would do]
- Short-term: [plan]
- Long-term: [automation opportunity]

WHAT I WOULD AUTOMATE:
- Auto-detect this condition via alert threshold
- Auto-notify via webhook/email
- Auto-document in ticket system
- Script to [specific action]
```

STOP TIMER.

DELIVERABLE: Completed incident notes document.

DAY 13 - OPS STORYTELLING

Time required: 2 hours

OBJECTIVE: Convert raw metrics into narrative. Build an RCA template.

STEP-BY-STEP ACTIONS:

1. **Understand the storytelling framework:**

Every ops narrative has four parts:

• **What happened:** The facts. Timeline. Metrics. What triggered.

• **Impact:** Who was affected. How many users. What services. How long.

• **Fix:** What was done to resolve it. By whom. How long it took.

• **Prevent:** What changes to prevent recurrence. Automation. Monitoring. Architecture.

2. **Practice converting raw data to narrative:**

Raw data: "Cluster-Prod-01 CPU hit 95% at 14:32, alert fired at 14:37, resolved at 15:15 after vMotion of 8 VMs."

Narrative: "On Tuesday at 2:32 PM, production cluster 01 experienced CPU saturation reaching 95%, triggered by a batch job on 3 oversized VMs. The automated alert fired at 2:37 PM and was acknowledged by the on-call engineer at 2:40 PM. The engineer identified the root cause as 3 VMs running unscheduled batch jobs that consumed 45% of cluster CPU. Eight VMs were migrated to cluster 02 via vMotion to relieve pressure, resolving the issue at 3:15 PM. Total user impact: approximately 200 users experienced application latency for 43 minutes. Prevention: the batch VMs have been moved to a dedicated resource pool with CPU limits, and a capacity alert has been set at 85% to provide earlier warning."

3. **BUILD YOUR DELIVERABLE - RCA Template:**

```
ROOT CAUSE ANALYSIS (RCA) TEMPLATE
====================================

INCIDENT ID: [INC-YYYY-NNNN]
DATE: [date]
SEVERITY: [P1/P2/P3]
DURATION: [start time] to [end time] ([total minutes])
AUTHOR: [your name]

1. EXECUTIVE SUMMARY (2-3 sentences)
[What happened, who was affected, how it was resolved.]
Example: "Production cluster CPU saturation caused application latency
for approximately 200 users over 43 minutes. Root cause was unscheduled
batch jobs on oversized VMs. Resolved by VM migration and resource pool
isolation."

2. TIMELINE
| Time | Event |
|---|---|
| 14:32 | CPU usage on Cluster-Prod-01 exceeds 90% |
| 14:37 | VCF Operations alert fires (Critical: CPU Contention) |
| 14:40 | On-call engineer acknowledges alert |
| 14:45 | Engineer identifies 3 VMs running batch jobs |
| 14:50 | Decision: vMotion 8 VMs to Cluster-Prod-02 |
| 15:10 | vMotion complete, CPU drops to 62% |
| 15:15 | Monitoring confirms normal performance, incident closed |

3. ROOT CAUSE
[Specific technical cause]
Example: "Three VMs (batch-proc-01, batch-proc-02, batch-proc-03) were
configured with 16 vCPU each and ran unscheduled data processing jobs
simultaneously, consuming 45% of cluster CPU capacity."

4. IMPACT
- Users affected: [number]
- Services affected: [list]
- Duration of impact: [minutes]
- Business impact: [description - revenue loss, SLA breach, etc.]

5. RESOLUTION
- Immediate: [what was done to stop the bleeding]
- Technical: [specific commands, actions, changes made]

6. PREVENTION (5 WHYS)
Why did CPU saturate? → 3 VMs ran batch jobs
Why were batch jobs running? → No scheduling control
Why no scheduling control? → VMs not in a resource pool with limits
Why no resource pool? → Batch workloads not identified in capacity planning
Why not identified? → No workload classification process

CORRECTIVE ACTIONS:
| Action | Owner | Due Date | Status |
|---|---|---|---|
```

```
| Move batch VMs to dedicated resource pool | [name] | [date] | |
| Set CPU limit on batch resource pool | [name] | [date] | |
| Add 85% CPU alert threshold to all prod clusters | [name] | [date] | |
| Create workload classification process | [name] | [date] | |
| Schedule batch jobs for off-peak hours | [name] | [date] | |

7. LESSONS LEARNED
- [What worked well in the response]
- [What could be improved]
- [What we will automate]
```

VALIDATION CHECK:

• Q: "Tell me about a time you identified and resolved a performance issue." A: Use the narrative format above. Tell the story with timeline, impact, fix, prevent.

DAY 14 - CHECKPOINT #2

Time required: 2 hours

YOU MUST PRESENT: A weekly ops briefing using your dashboards (5 minutes).

EXACT SCRIPT FOR YOUR 5-MINUTE BRIEFING:

Stand up (even if alone). Open your dashboards. Speak this out loud:

[Minute 0:00-1:00] - Opening + Overall Health

"Good morning. This is the weekly infrastructure operations briefing for the week of [date range]. Overall health status: [green/yellow/red]. We have [N] active critical alerts and [N] immediate alerts. No P1 incidents this week. [or: We had one P1 incident on [day], which I will cover in a moment.]"

[Minute 1:00-2:00] - Capacity Status

"Switching to capacity. All production clusters are above 30% remaining, which is our comfort threshold. Dev cluster 2 is trending toward exhaustion in [N] days. I have submitted a right-sizing request for 12 oversized VMs that would reclaim approximately [N] GB of memory. Reclaimable storage from old snapshots: [N] GB."

[Minute 2:00-3:00] - Performance + Incidents

"Performance highlights: We saw a CPU spike on Prod Cluster 01 on Tuesday at 2:32 PM. Root cause was unscheduled batch jobs. Resolution took 43 minutes. Full RCA is documented and corrective actions are in progress. No other performance events exceeded thresholds this week."

[Minute 3:00-4:00] - Compliance + Risks

"Compliance status: [N] hosts are fully compliant. [N] hosts have findings: [list top findings, e.g., SSH enabled on 2 hosts, NTP drift on 1 host]. Remediation is scheduled for [date]. Top risk this week: Dev cluster 2 capacity. Second risk: certificate expiring on [component] in [N] days."

[Minute 4:00-5:00] - Actions + Questions

"Action items for this week: One, right-size 12 VMs on Dev cluster 2. Two, remediate SSH findings on 2 hosts. Three, renew certificate for [component]. That concludes the briefing. Questions?"

WEEK 3: DAYS 15-21 (Logs Mastery: VCF Operations for Logs + Analyze + Dashboards/Alerts/Queries)

IMPORTANT VCF 9.x CONTEXT: In VCF 9.0, log analysis is integrated into VCF Operations under **Infrastructure Operations > Analyze**. The Analyze section provides log search, saved queries, extracted fields, event types, trends, and side-by-side query comparison. This requires the **VCF Operations for Logs** component to be deployed. Log data is standardized to RFC 5424 format. You can migrate data from legacy Aria Operations for Logs 8.x (up to 90 days).

DAY 15 - WHAT LOGS ARE FOR (TROUBLESHOOTING VS AUDITING)

Time required: 2 hours

OBJECTIVE: Define log use cases and required log sources.

STEP-BY-STEP ACTIONS:

1. **Understand the two primary log use cases:**

Troubleshooting logs (real-time, reactive):

• Purpose: Find root cause of current issues

• Sources: ESXi /var/log/vmkernel.log, vCenter vpxd.log, NSX syslog, SDDC Manager logs

• Retention: 7-30 days (recent, detailed)

• Questions answered: "Why did this host disconnect?" "Why did this VM crash?" "What error caused the deployment to fail?"

Audit logs (historical, proactive):

• Purpose: Track who did what, when, for security and compliance

• Sources: vCenter events/tasks, ESXi shell access logs, NSX audit logs, SSO login logs

• Retention: 90 days to 1 year (longer, may be summarized)

• Questions answered: "Who logged into this host via SSH?" "Who changed this firewall rule?" "Who deleted that VM?"

2. **Build your log source inventory checklist:**

```
LOG SOURCE INVENTORY CHECKLIST
================================

SOURCE 1: ESXi Hosts
  Log files:
  - /var/log/vmkernel.log (kernel messages, storage, network)
  - /var/log/hostd.log (host agent, VM operations)
  - /var/log/vpxa.log (vCenter agent on host)
  - /var/log/vobd.log (VMware Observation Broker - events)
  - /var/log/auth.log (authentication, SSH access)
  - /var/log/shell.log (ESXi Shell commands executed)
  Forwarding to Operations-Logs: YES/NO
  Forwarding method: syslog (UDP 514 or TCP 514 or TCP 1514)

SOURCE 2: vCenter Server
  Log files:
  - /var/log/vmware/vpxd/vpxd.log (core vCenter service)
  - /var/log/vmware/vpxd/vpxd-alert.log (critical alerts)
  - /var/log/vmware/sso/ssoAdminServer.log (SSO/authentication)
```

```
  - /var/log/vmware/content-library/ (content library)
  Forwarding to Operations-Logs: YES/NO
  Events database: Task & Event Manager (retained in DB)

SOURCE 3: NSX Manager
  Log files:
  - /var/log/syslog (general NSX operations)
  - /var/log/proton/nsxapi.log (API calls)
  - /var/log/corfu/corfu.log (distributed datastore)
  Audit events: NSX audit log (rule changes, policy updates)
  Forwarding to Operations-Logs: YES/NO

SOURCE 4: SDDC Manager
  Log files:
  - /var/log/vmware/vcf/sddc-manager/ (lifecycle operations)
  - /var/log/vmware/vcf/domainmanager/ (domain operations)
  - /var/log/vmware/vcf/operationsmanager/ (ops)
  Forwarding to Operations-Logs: YES/NO

SOURCE 5: VCF Operations (itself)
  Log files:
  - /var/log/vmware/vcops/ (analytics engine)
  - Collector logs
  Forwarding: Self-ingestion or separate log target

FORWARDING PROTOCOL SETTINGS:
  Protocol: TCP (recommended) or UDP
  Port: 514 (standard) or 1514 (non-standard)
  Format: RFC 5424 (preferred) or RFC 3164
  TLS: YES (if supported)
```

DELIVERABLE: Completed log source inventory checklist for your environment.

DAY 16 - DEPLOY VCF OPERATIONS FOR LOGS (OPERATIONS-LOGS)

Time required: 3-4 hours

OBJECTIVE: Follow fleet-managed deployment workflow. Understand all required inputs.

STEP-BY-STEP ACTIONS:

1. **Pre-deployment checklist (verify ALL of these before starting):**

• DNS forward and reverse records created for the Operations-Logs appliance FQDN

• NTP server accessible from the Operations-Logs network

• IP address allocated (static, not DHCP)

• Network connectivity: Operations-Logs must reach all log sources

• Sufficient storage: plan for log volume (rule of thumb: 1 GB/day per 10 hosts for 30-day retention)

• Certificate: either use auto-generated or prepare a custom CA-signed certificate

• Admin credentials decided (write them down securely)

2. **Deploy via SDDC Manager (fleet-managed deployment):**

• Log into SDDC Manager UI: `https://<sddc-mgr-fqdn>/ui`

• Navigate to **Lifecycle Management** or **Inventory** > **Workload Domains**

• Select the Management Domain

• Look for the option to deploy **VCF Operations for Logs** (Operations-Logs)

• Click **Deploy** or **Add**

3. **Fill in deployment parameters (exact fields you will see):**

• **Appliance FQDN:** `ops-logs.yourdomain.com` (must match DNS record you created)

• **IP Address:** `10.0.0.50` (your allocated static IP)

• **Subnet Mask:** `255.255.255.0`

• **Gateway:** `10.0.0.1`

• **DNS Server(s):** `10.0.0.10` (your DNS server)

• **NTP Server(s):** `10.0.0.11` (your NTP server)

• **Admin Password:** [your secure password]

• **Certificate:** Select auto-generated or upload custom

• **Deployment Size:** Small / Medium / Large (based on environment size)

• Small: up to 20 hosts

• Medium: up to 100 hosts

• Large: 100+ hosts

4. **Review and submit deployment:**

• Double-check every field

• Click **Submit** or **Deploy**

• The deployment will take 30-60 minutes

5. **Monitor deployment progress:**

• In SDDC Manager, watch the task progress

• Wait for status = Completed Successfully

6. **Post-deployment validation:**

• Open the Operations-Logs UI: `https://<ops-logs-fqdn>`

• Log in with admin credentials

• Verify the dashboard loads

• Go to log sources - verify at least one source is connected

• Check log ingestion: are logs flowing in?

7. **Configure log forwarding from ESXi hosts (if not automatic):**

• In vCenter, go to each host > Configure > System > Advanced System Settings

• Set: `Syslog.global.logHost = tcp://<ops-logs-fqdn>:514`

• Or use PowerCLI to set all hosts at once (you will learn this in Week 4)

DELIVERABLE: Deployment validation checklist (all items checked):

```
DEPLOYMENT VALIDATION CHECKLIST
================================
[ ] DNS forward record resolves: nslookup ops-logs.yourdomain.com → correct IP
[ ] DNS reverse record resolves: nslookup <IP> → ops-logs.yourdomain.com
[ ] NTP synchronized: appliance time matches reference
[ ] UI accessible: https://<ops-logs-fqdn> loads login page
[ ] Admin login works: can log in with configured credentials
[ ] Log sources visible: at least one source showing in configuration
[ ] Logs flowing: new log entries appearing in real-time view
[ ] Certificate valid: browser shows valid certificate (no warnings)
```

DAY 17 - CONTENT PACKS & PREBUILT DASHBOARDS/ALERTS/QUERIES

Time required: 2 hours

OBJECTIVE: Use prebuilt content to accelerate operations. Identify top 10 dashboards to pin.

STEP-BY-STEP ACTIONS:

1. Log into Operations-Logs UI.

2. Navigate to **Content Packs** (or Administration > Content Packs):
• Content packs are pre-built bundles containing dashboards, alerts, and saved queries
• They are designed for specific use cases (vSphere, NSX, vSAN, etc.)
• Using them saves you from building everything from scratch

3. Browse available content packs:
• **VMware - vSphere:** ESXi and vCenter log dashboards
• **VMware - NSX:** Network and security log dashboards
• **VMware - vSAN:** Storage log dashboards
• **General:** Linux syslog, authentication, etc.

4. Install/enable content packs relevant to your environment:
• Click on each content pack
• Click **Install** or **Enable**
• Note what dashboards, alerts, and queries each pack includes

5. **BUILD YOUR DELIVERABLE - Top 10 Dashboards to Pin:**

```
TOP 10 LOG DASHBOARDS TO PIN
============================

1. ESXi Overview
   What it shows: All ESXi host log events, error rates, warning trends
   Why pin it: First stop for host troubleshooting

2. vCenter Events
   What it shows: vCenter tasks and events, who did what
   Why pin it: Audit trail and change tracking

3. Authentication Failures
   What it shows: Failed login attempts across all components
   Why pin it: Security monitoring, brute force detection

4. ESXi Error Trends
   What it shows: Error log volume over time, top error types
   Why pin it: Spot increasing failure patterns before outages

5. NSX Firewall Rule Changes
   What it shows: When firewall rules were added/modified/deleted
   Why pin it: Security audit, unauthorized change detection

6. Storage Errors
   What it shows: SCSI errors, path failures, vSAN issues
   Why pin it: Early warning for storage failures

7. VM Power Events
   What it shows: VM power on/off/reset/vMotion events
```

```
   Why pin it: Track unexpected VM restarts

8. Certificate Warnings
   What it shows: Certificate expiration warnings from all components
   Why pin it: Prevent outages from expired certificates

9. SDDC Manager Operations
   What it shows: Lifecycle operations, deployment tasks, update progress
   Why pin it: Track VCF lifecycle health

10. Syslog Ingestion Health
    What it shows: Log ingestion rate, dropped logs, source connectivity
    Why pin it: Ensure your logging infrastructure itself is healthy
```

DAY 18 - CREATE LOG DASHBOARDS (OPERATIONAL VIEWS)

Time required: 3 hours

OBJECTIVE: Build custom log dashboards for operational use.

STEP-BY-STEP ACTIONS:

1. Log into Operations-Logs UI.

2. **Build Dashboard 1: Error Trends**

• Go to Dashboards > Create New

• Name: `NOC-Logs - Error Trends`

• Add a time-series chart:

• Query: filter logs where severity = ERROR or CRITICAL

• Group by: source (host/vCenter/NSX)

• Time range: Last 24 hours

• This shows you error volume over time, per source

• Add a table widget:

• Query: Top 20 most frequent error messages (last 24h)

• Columns: Count, Error Message, Source

• Save

3. **Build Dashboard 2: Authentication Failures**

• Create: `SEC-Logs - Auth Failures`

• Add a time-series chart:

• Query: filter logs where message contains "authentication failure" OR "login failed" OR "Access denied"

• Group by: source IP or username

• Time range: Last 7 days

• Add a table widget:

• Query: Failed login attempts with username, source IP, target system, timestamp

• Sort by: newest first

• Save

4. **Build Dashboard 3: vCenter Alarm History**

• Create: `OPS-Logs - vCenter Alarms`

• Add widgets showing:

• Alarm triggered events over time

• Top alarm types by frequency

• Alarm cleared events (to show resolution)

• Save

5. **Document your "Logs NOC Board" spec:**

```
LOGS NOC BOARD DASHBOARD SPECIFICATION
======================================

Dashboard Name: NOC-Logs - Operations Overview
Purpose: Single pane for NOC operator to monitor log health

Layout:
Row 1: Error Trend (time series, 24h) | Log Ingestion Rate (time series, 24h)
Row 2: Auth Failures (last 4h, table) | Top Error Messages (last 4h, table)
Row 3: Recent Critical Events (table, last 1h, newest first) — full width

Filters available: Time range selector, Source filter (ESXi/vCenter/NSX/SDDC)
Auto-refresh: Every 60 seconds
```

DAY 19 - LOG-BASED ALERT PHILOSOPHY

Time required: 2 hours

OBJECTIVE: Define "alert on symptom, not every event." Build alert playbook entries.

STEP-BY-STEP ACTIONS:

1. **Understand the philosophy:**

• BAD: Alert on every ERROR log line → thousands of alerts, all noise

• GOOD: Alert on SYMPTOMS → "Error rate for host X exceeded 50 errors/hour" or "3 or more auth failures for same user in 5 minutes"

• The principle: aggregate, threshold, then alert

2. **BUILD YOUR DELIVERABLE - Alert Playbook:**

```
LOG-BASED ALERT PLAYBOOK
========================

ALERT 1: Authentication Brute Force Suspected
  Trigger: 5+ failed login attempts for the same username within 10 minutes
  Severity: P2 - Immediate
  Verify:
    1. Open Auth Failures dashboard
    2. Filter by the username from the alert
    3. Check source IPs - is it one IP or many?
    4. Check if the account eventually succeeded (compromised?)
  Remediate:
    - If one source IP: Block IP at NSX firewall, disable account temporarily
    - If many source IPs: Disable account, reset password, notify user
    - If account succeeded after failures: Treat as compromised, force password reset, review activity
  Escalate to: Security team if account was compromised
```

```
ALERT 2: ESXi Host Error Rate Spike
  Trigger: More than 100 ERROR-level log entries from a single ESXi host in 1 hour (baseline is <10/hour
)
  Severity: P3 - Warning (escalate to P2 if sustained >2 hours)
  Verify:
    1. Open Error Trends dashboard
    2. Filter to the specific host
    3. Read the actual error messages - are they storage, network, or hardware?
    4. Check VCF Operations for correlated health alerts on this host
  Remediate:
    - Storage errors: Check HBA paths, datastore connectivity
    - Network errors: Check vmnic status, DVS configuration
    - Hardware errors: Check hardware health via iLO/iDRAC, open vendor case
  Escalate to: Hardware team if physical component failure suspected

ALERT 3: vCenter Service Restart Detected
  Trigger: Log entry matching "vpxd started" or "service restarted" outside maintenance window
  Severity: P2 - Immediate
  Verify:
    1. Open vCenter Events dashboard
    2. Check if a planned maintenance was scheduled (check change calendar)
    3. Check if vpxd crashed (look for crash dump logs before the restart)
    4. Check VCF Operations for vCenter health alerts
  Remediate:
    - If planned: Verify service is healthy post-restart, close alert
    - If unplanned crash: Collect logs, check for known issues, open support case if recurring
  Escalate to: VMware support if crash is recurring

ALERT 4: Log Ingestion Stopped
  Trigger: Zero logs received from any configured source for >15 minutes
  Severity: P2 - Immediate
  Verify:
    1. Open Ingestion Health dashboard
    2. Identify which source(s) stopped sending
    3. Ping the source - is it reachable?
    4. SSH to source - is syslog service running?
  Remediate:
    - Source unreachable: Network issue or host down → check VCF Operations
    - Syslog service stopped: Restart syslog service
    - Operations-Logs appliance issue: Check appliance health, restart ingestion service
  Escalate to: Infrastructure team if source is down

ALERT 5: NSX Firewall Rule Modified Outside Change Window
  Trigger: Log entry matching firewall rule create/modify/delete AND current time is outside approved ch
ange window (e.g., outside Tue/Thu 6PM-10PM)
  Severity: P1 - Critical
  Verify:
    1. Open NSX Firewall Rule Changes dashboard
    2. Identify who made the change (username)
    3. Identify what rule was changed
    4. Check change management system for approved change ticket
  Remediate:
    - If authorized (emergency change with verbal approval): Document retroactively
    - If unauthorized: Revert the change immediately, disable the user account, notify security team
  Escalate to: Security team and change management immediately
```

DAY 20 - INCIDENT DAY #2 (TIMED)

Time required: 2 hours

SIMULATION: "It is 8:45 AM Monday. A user reports: 'I cannot log into vCenter. The page loads but after entering credentials, I get an error.'"

SET A 30-MINUTE TIMER.

Triage steps:

Minutes 0-3: Reproduce and Scope

1. Try logging into vCenter yourself: `https://<vcenter-fqdn>/ui`
2. Can YOU log in? If yes → user-specific issue. If no → service issue.
3. Try SSO login at: `https://<vcenter-fqdn>/websso`
4. Write down: "Can I log in: YES/NO. Error message: [exact text]"

Minutes 3-8: Check VCF Operations

5. Log into VCF Operations (this should work independently of vCenter SSO issues if using local auth)
6. Check: Is there an alert for vCenter or SSO?
7. Check: vCenter health badge - is it green?
8. Write down: "vCenter health: [status]. Active alerts: [list]"

Minutes 8-15: Check Logs

9. Open Operations-Logs
10. Search for: vCenter SSO logs in the last 1 hour
11. Filter by: "authentication" OR "login" OR "SSO" OR "error"
12. Look for: specific error messages around the reported time (8:45 AM)
13. Common findings:

• "Token validation error" → SSO service issue

• "User not found" → directory sync issue

• "Password expired" → user password issue

• "Connection refused to Identity Provider" → AD/LDAP connectivity issue

14. Write down: "Log evidence: [exact error message and timestamp]"

Minutes 15-22: Root Cause Determination

15. Based on logs, determine the cause:

• If SSO service crashed: Check vpxd-alert.log for crash evidence

• If AD/LDAP unreachable: Check network connectivity from vCenter to AD (DNS, port 389/636)

• If certificate expired: Check vCenter certificate dates

• If user-specific: Check user account in SSO or AD

16. Write down: "Root cause: [specific finding]"

Minutes 22-28: Resolution

17. Based on root cause:

• SSO service down: SSH to vCenter, run `service-control --status --all`, restart SSO: `service-control --restart vmware-stsd`

• AD unreachable: Fix DNS or network path to AD server

• Cert expired: Follow VMware cert renewal process

• User account locked: Unlock in AD or SSO admin

18. Write down: "Resolution: [what you did]"

Minutes 28-30: Document

19. Complete incident notes (same format as Day 12)

STOP TIMER.

DAY 21 - CHECKPOINT #3

YOU MUST DEMO: A log dashboard + show a query path that proves root cause.

EXACT DEMO SCRIPT (5 minutes):

"I am going to demonstrate how I use log dashboards to prove root cause for a vCenter login failure.

[Open Operations-Logs UI]

Step 1: I open my Auth Failures dashboard. Here I can see all failed authentication attempts across the environment. I filter to the last 2 hours and to vCenter as the source.

[Point to the data on screen]

Step 2: I see a cluster of failed attempts starting at 8:43 AM. The error message is [read the actual error]. This tells me the SSO service was returning token validation errors.

Step 3: I pivot to a broader search. I search for 'vmware-stsd' in the last 2 hours. I find a log entry at 8:41 AM showing the STS service restarted unexpectedly. This is 2 minutes before the user reported the issue.

Step 4: I search for what happened before the restart. I filter to 8:30-8:41 AM on the vCenter source. I find out-of-memory errors in the STS process at 8:39 AM.

Step 5: Root cause proven. The STS service ran out of memory, crashed, restarted, and during the restart window users could not authenticate. The fix is to increase the STS service memory allocation and monitor for memory growth.

That is the query path: Symptom (login failures) → Authentication logs → Service crash → Memory exhaustion. Each step is provable with log evidence."

WEEK 4: DAYS 22-30 (API + Automation + Capstone)

DAY 22 - VCF OPERATIONS API: SWAGGER & TOOLS

Time required: 2-3 hours

OBJECTIVE: Find Swagger UI and understand how to use it. Create an API quickstart page.

STEP-BY-STEP ACTIONS:

1. **Open Swagger UI:**

• In your browser, navigate to: `https://<your-ops-fqdn>/suite-api/doc/swagger-ui.html`

• Example: `https://vrops.lab.local/suite-api/doc/swagger-ui.html`

• This page lists EVERY API endpoint available in VCF Operations

2. **Explore the API categories:**

• Scroll through the page. You will see groups like:

• **Authentication** (token acquire/release)

• **Alerts** (list, acknowledge, close alerts)

• **Resources** (inventory objects)

• **Stats** (metrics data)

• **Dashboards** (CRUD operations on dashboards)

• **Reports** (generate/download reports)

• Click on any group to expand it and see individual endpoints

3. **Try a simple call in Swagger UI:**

• First you need to authenticate (you will learn this in detail on Day 23)

• Find the **Authentication** section

• Click on `POST /api/auth/token/acquire`

• Click **Try it out**

• In the request body, enter:

```
{
  "username": "admin",
  "password": "your-password",
  "authSource": "local"
}
```

• Click **Execute**

• You should get a 200 response with a token in the response body

• Copy the token value

4. **Use the token to make an API call:**

• Find the **Alerts** section

• Click on `GET /api/alerts`

• Click **Try it out**

• In the Authorization field, enter: `OpsToken <your-token>`

• Click **Execute**

• You should get a JSON response with alert data

5. **BUILD YOUR DELIVERABLE - API Quickstart Page:**

```
VCF OPERATIONS API QUICKSTART
==============================

SWAGGER UI URL:
https://<ops-fqdn>/suite-api/doc/swagger-ui.html

BASE API URL:
https://<ops-fqdn>/suite-api/api

AUTHENTICATION:
  Endpoint: POST /api/auth/token/acquire
  Body: {"username": "...", "password": "...", "authSource": "local"}
  Response: {"token": "abc123...", "validity": 21600, ...}
  Use token: Authorization: OpsToken abc123...

COMMON ENDPOINTS:
```

```
  GET  /api/alerts                    → List all alerts
  GET  /api/alerts/{alertId}          → Get specific alert
  GET  /api/resources                 → List all resources (objects)
  GET  /api/resources/{resourceId}    → Get specific resource
  GET  /api/resources/{id}/stats      → Get metrics for a resource
  GET  /api/dashboards                → List all dashboards
  POST /api/reports                   → Generate a report

RESPONSE FORMAT: JSON (set Accept: application/json header)

TOOLS TO USE:
  Browser: Swagger UI (learning + quick tests)
  Postman: Collection-based testing + environment variables
  cURL: Command-line scripting
  Python: requests library for automation scripts
  PowerShell: Invoke-RestMethod for operational scripts

RATE LIMITS / BEST PRACTICES:
  - Tokens are valid for ~6 hours by default
  - Always release tokens when done: POST /api/auth/token/release
  - Use pagination for large result sets (pageSize, page parameters)
  - Use filters to narrow results (avoid pulling all objects every time)
```

DAY 23 - AUTH PATTERNS (OPSTOKEN / SSO TOKEN)

Time required: 2 hours

OBJECTIVE: Master token acquisition and header formats.

STEP-BY-STEP ACTIONS:

1. **Method 1: Local auth (OpsToken)**

• Open Postman (or cURL in terminal)

• Create a new POST request:

• URL: `https://<ops-fqdn>/suite-api/api/auth/token/acquire`

• Headers: `Content-Type: application/json`, `Accept: application/json`

• Body (raw JSON):

```
{
  "username": "admin",
  "password": "YourPassword123!",
  "authSource": "local"
}
```

• Send the request

• Response will look like:

```
{
  "token": "e1c2f5a8-...-long-token-string",
  "validity": 21600,
  "expiresAt": "Thursday, March 20, 2026 6:00:00 PM UTC",
  "roles": []
}
```

• Copy the `token` value

2. **Use the token in subsequent requests:**

• For every API call after authentication, add this header:

- `Authorization: OpsToken e1c2f5a8-...-long-token-string`
- Example GET request:
- URL: `https://<ops-fqdn>/suite-api/api/alerts`
- Headers:
- `Authorization: OpsToken e1c2f5a8-...-long-token-string`
- `Accept: application/json`
- Send → you get alert data back

3. **Release the token when done:**

- POST: `https://<ops-fqdn>/suite-api/api/auth/token/release`
- Headers: `Authorization: OpsToken e1c2f5a8-...-long-token-string`
- This invalidates the token (security best practice)

4. **Method 2: SSO / vIDM auth (if configured):**

- If your environment uses SSO/vIDM integration:
- `authSource` value changes to your identity source name (e.g., `"vsphere.local"`)
- Token format and header remain the same: `Authorization: OpsToken <token>`

5. **BUILD YOUR DELIVERABLE - Auth Cheat Sheet:**

```
VCF OPERATIONS API AUTH CHEAT SHEET
====================================

--- LOCAL AUTH ---
POST https://<ops>/suite-api/api/auth/token/acquire
Content-Type: application/json
Accept: application/json

Body:
{
  "username": "admin",
  "password": "P@ssw0rd",
  "authSource": "local"
}

Response: {"token": "<TOKEN>", "validity": 21600, ...}

--- USE TOKEN ---
GET https://<ops>/suite-api/api/alerts
Authorization: OpsToken <TOKEN>
Accept: application/json

--- RELEASE TOKEN ---
POST https://<ops>/suite-api/api/auth/token/release
Authorization: OpsToken <TOKEN>

--- SSO AUTH ---
Same as above but authSource = "vsphere.local" (or your identity source name)

--- CURL EXAMPLES ---

# Acquire token:
curl -k -X POST "https://<ops>/suite-api/api/auth/token/acquire" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{"username":"admin","password":"P@ssw0rd","authSource":"local"}'

# Use token (get alerts):
curl -k -X GET "https://<ops>/suite-api/api/alerts" \
```

```
  -H "Authorization: OpsToken <TOKEN>" \
  -H "Accept: application/json"

# Release token:
curl -k -X POST "https://<ops>/suite-api/api/auth/token/release" \
  -H "Authorization: OpsToken <TOKEN>"

--- COMMON MISTAKES ---
1. Forgetting "OpsToken " prefix (with the space) before the token
2. Token expired (default 6 hours) - acquire a new one
3. Wrong authSource value - check your identity source name
4. Not setting Accept: application/json - may get XML instead
5. Using HTTP instead of HTTPS - API requires TLS
```

DAY 24 - BUILD YOUR POSTMAN COLLECTION

Time required: 2-3 hours

OBJECTIVE: Import OpenAPI specs and build a working Postman collection.

STEP-BY-STEP ACTIONS:

1. **Open Postman** (download from postman.com if needed, it is free)

2. **Create a new Environment:**

• Click the gear icon (Environments) in Postman

• Click **Add**

• Name: `VCF Ops - Lab`

• Add variables:

• `ops_host` = `your-ops-fqdn` (e.g., `vrops.lab.local`)

• `ops_token` = (leave blank, will be populated by auth request)

• `ops_user` = `admin`

• `ops_pass` = `YourPassword` (mark as "secret" type)

• Click **Save**

• Select this environment as active (dropdown in top-right of Postman)

3. **Import the OpenAPI spec (if available):**

• If you have the VCF Operations OpenAPI spec file (JSON or YAML):

• Click **Import** in Postman

• Drag in the spec file or paste the URL

• Postman generates an entire collection with all endpoints pre-configured

• If you do not have the spec file:

• You can manually build the collection (steps below)

• Or download from Broadcom developer portal / GitHub VCF API specs repo

4. **Build the collection manually (if needed):**

• Click **New** > **Collection**

• Name: `VCF Ops - Core`

- Add folders inside the collection:
- `Auth`
- `Alerts`
- `Resources`
- `Stats`
- `Dashboards`

5. **Add the Auth request:**

- In the `Auth` folder, click **Add Request**
- Name: `Acquire Token`
- Method: POST
- URL: `https://{{ops_host}}/suite-api/api/auth/token/acquire`
- Headers: `Content-Type: application/json`, `Accept: application/json`
- Body (raw JSON):

```
{
  "username": "{{ops_user}}",
  "password": "{{ops_pass}}",
  "authSource": "local"
}
```

- **Tests** tab (this auto-saves the token for other requests):

```
var jsonData = pm.response.json();
pm.environment.set("ops_token", jsonData.token);
```

- Save

6. **Add Alert requests:**

- In `Alerts` folder, add:
- `List All Alerts`: GET `https://{{ops_host}}/suite-api/api/alerts`
- Header: `Authorization: OpsToken {{ops_token}}`
- `Get Alert by ID`: GET `https://{{ops_host}}/suite-api/api/alerts/{{alert_id}}`
- Header: `Authorization: OpsToken {{ops_token}}`

7. **Add Resource requests:**

- In `Resources` folder, add:
- `List Resources`: GET `https://{{ops_host}}/suite-api/api/resources`
- Header: `Authorization: OpsToken {{ops_token}}`
- `Get Resource Stats`: GET `https://{{ops_host}}/suite-api/api/resources/{{resource_id}}/stats`
- Header: `Authorization: OpsToken {{ops_token}}`

8. **Test the collection:**

- First, send `Acquire Token` → verify 200 response and token saved
- Then send `List All Alerts` → verify 200 response with alert JSON
- Then send `List Resources` → verify 200 response with resource JSON

DELIVERABLE: Working Postman collection with Auth, Alerts, Resources folders.

DAY 25 - VCF SDK AWARENESS (PYTHON/JAVA) + SAMPLES

Time required: 2 hours

OBJECTIVE: Know what SDKs exist, where to find them, and when to use each tool.

STEP-BY-STEP ACTIONS:

1. **Identify the SDK options:**

• **VCF Unified SDK (Python):** `pip install vcf-sdk` (or from Broadcom developer portal)

• Structured Python library for VCF component APIs

• Best for: writing automation scripts, integrations, custom tools

• **VCF Unified SDK (Java):** Maven/Gradle dependency

• Best for: enterprise applications, CI/CD integrations

• **OpenAPI Specs (language-agnostic):** JSON/YAML files

• Can generate client libraries in ANY language

• Best for: Postman collections, custom client generation (openapi-generator)

• **VCF PowerCLI:** PowerShell module

• Best for: operational scripting, admin tasks, batch operations

• Familiar to VMware admins

2. **Find SDK samples:**

• Check Broadcom developer portal for VCF SDK documentation

• Check GitHub for VMware/Broadcom VCF SDK sample repositories

• Look for sample scripts covering:

• Authentication

• Listing resources

• Getting metrics

• Creating/managing dashboards

• Managing alerts

3. **BUILD YOUR DELIVERABLE - Automation Path Chooser:**

```
AUTOMATION PATH CHOOSER
=========================

QUESTION: "Which tool should I use?"

USE VCF POWERCLI WHEN:
  - You are a PowerShell admin (most VMware admins are)
  - You need quick operational scripts (get health, list alerts, tag objects)
  - You are doing one-off or scheduled tasks
  - You want tab-completion and discoverability
  - Example: "Get all hosts in maintenance mode and export to CSV"

USE VCF PYTHON SDK WHEN:
  - You are building a reusable automation tool or integration
  - You need complex logic (conditionals, error handling, retries)
  - You want to integrate with other Python tools (Ansible, Flask, Django)
  - You are building a custom dashboard or reporting tool
  - Example: "Build a daily health report that emails the team"

USE OPENAPI + POSTMAN WHEN:
```

```
  - You are learning the API (fastest way to explore endpoints)
  - You need to test a specific API call quickly
  - You want to generate client libraries in any language
  - You are documenting API workflows for your team
  - Example: "Figure out what parameters the alert endpoint accepts"

USE CURL WHEN:
  - You need a quick one-liner from the command line
  - You are scripting in bash
  - You are troubleshooting API issues (raw HTTP visibility)
  - Example: "Quick check if the API is responding"

USE RAW REST (requests library) WHEN:
  - The SDK does not cover a specific endpoint
  - You need maximum control over the HTTP request
  - You are working with a newer API version not yet in the SDK
  - Example: "Call a brand-new endpoint that was just released"

DECISION FLOWCHART:
  Am I exploring/learning? → Postman
  Am I an admin doing operational tasks? → PowerCLI
  Am I building a reusable tool? → Python SDK
  Am I doing a quick CLI check? → cURL
  Does the SDK not support what I need? → Raw REST
```

DAY 26 - VCF INSTALLER AUTOMATION LANDSCAPE

Time required: 2 hours

OBJECTIVE: Understand the VCF Installer's automation capabilities.

STEP-BY-STEP ACTIONS:

1. **Understand what VCF Installer automates:**

• VCF Installer (part of SDDC Manager) handles deployment of the entire VCF stack

• It exposes OpenAPI 3.0 specs for deployment workflows

• It integrates with PowerCLI for scripted deployments

• Key deployment workflows:

• Bring-up (initial VCF deployment)

• Workload domain creation

• Host commissioning/decommissioning

• Cluster expansion/contraction

2. **Identify automation opportunities:**

```
TOP 10 VCF AUTOMATION OPPORTUNITIES
====================================

1. Automated Health Check Export
   What: Script that pulls health status for all clusters and exports to CSV/JSON daily
   Tool: Python SDK or PowerCLI
   Value: Daily health snapshot without manual UI clicks

2. Alert Report Generation
   What: Script that pulls active P1/P2 alerts and emails the ops team
   Tool: Python SDK + email library
   Value: No one misses a critical alert
```

```
3. Capacity Report Automation
   What: Script that pulls capacity remaining for all clusters, calculates trend, generates report
   Tool: Python SDK
   Value: Weekly capacity report generated automatically

4. Host Commissioning Automation
   What: Script that commissions new ESXi hosts into SDDC Manager
   Tool: SDDC Manager API / PowerCLI
   Value: Consistent, repeatable host onboarding

5. Certificate Monitoring
   What: Script that checks all certificate expiration dates, alerts if <30 days
   Tool: Python + VCF Operations API
   Value: Never be surprised by an expired cert

6. Configuration Drift Check
   What: Script that compares current host configurations against baseline
   Tool: PowerCLI
   Value: Automated compliance checking

7. Snapshot Cleanup
   What: Script that finds and reports/deletes snapshots older than N days
   Tool: PowerCLI
   Value: Reclaim storage, prevent performance issues

8. VM Inventory Export
   What: Script that exports complete VM inventory with resource allocations
   Tool: PowerCLI or Python SDK
   Value: Asset management, chargeback data

9. Workload Domain Lifecycle
   What: Script that validates prerequisites before creating a workload domain
   Tool: SDDC Manager API
   Value: Fewer failed deployments

10. Password Rotation
    What: Script that rotates service account passwords per policy
    Tool: SDDC Manager API (password management endpoints)
    Value: Security compliance, audit readiness
```

DAY 27 - VCF POWERCLI FUNDAMENTALS

Time required: 3 hours

OBJECTIVE: Install VCF PowerCLI and write script skeletons.

STEP-BY-STEP ACTIONS:

1. **Install PowerCLI:**

• Open PowerShell as Administrator

• Run:

```
Install-Module -Name VMware.PowerCLI -Scope CurrentUser -Force
```

• If prompted about untrusted repository, type: `Y` and press Enter

• Verify installation:

```
Get-Module -Name VMware.PowerCLI -ListAvailable
```

2. **Configure PowerCLI settings:**

```
   # Ignore invalid certificates in lab environments:
   Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false

   # Set default server mode:
   Set-PowerCLIConfiguration -DefaultVIServerMode Multiple -Confirm:$false
```

3. **Connect to vCenter:**

```
   Connect-VIServer -Server "vcenter.yourdomain.com" -User "administrator@vsphere.local" -Password "Your
Password"
```

4. **Practice basic commands:**

```
   # List all clusters:
   Get-Cluster

   # List all hosts:
   Get-VMHost

   # List all VMs:
   Get-VM

   # Get host health info:
   Get-VMHost | Select-Object Name, ConnectionState, PowerState, NumCpu, MemoryTotalGB, MemoryUsageGB

   # Get VMs with CPU/Memory allocation:
   Get-VM | Select-Object Name, PowerState, NumCpu, MemoryGB, UsedSpaceGB

   # Find VMs with snapshots:
   Get-VM | Get-Snapshot | Select-Object VM, Name, Created, SizeGB
```

5. **BUILD YOUR DELIVERABLE - PowerCLI Script Skeleton Set:**

```
# ============================================================
# SCRIPT: Get-VcfOpsHealth.ps1
# PURPOSE: Export cluster health summary to CSV
# ============================================================

param(
    [Parameter(Mandatory=$true)]
    [string]$VCenterServer,

    [Parameter(Mandatory=$true)]
    [string]$Username,

    [string]$OutputPath = ".\cluster_health_$(Get-Date -Format 'yyyyMMdd').csv"
)

# Connect
Connect-VIServer -Server $VCenterServer -User $Username -Password (Read-Host -AsSecureString "Password")

# Gather data
$clusters = Get-Cluster | ForEach-Object {
    $cluster = $_
    $hosts = Get-VMHost -Location $cluster
    $vms = Get-VM -Location $cluster

    [PSCustomObject]@{
        ClusterName     = $cluster.Name
        HostCount       = $hosts.Count
        VMCount         = $vms.Count
        TotalCPUGHz     = [math]::Round(($hosts | Measure-Object -Property CpuTotalMhz -Sum).Sum / 1000,
 2)
        UsedCPUGHz      = [math]::Round(($hosts | Measure-Object -Property CpuUsageMhz -Sum).Sum / 1000,
 2)
        TotalMemoryGB   = [math]::Round(($hosts | Measure-Object -Property MemoryTotalGB -Sum).Sum, 2)
        UsedMemoryGB    = [math]::Round(($hosts | Measure-Object -Property MemoryUsageGB -Sum).Sum, 2)
```

```
        CPUUsagePct     = 0  # calculated below
        MemUsagePct     = 0  # calculated below
    }
}

# Calculate percentages
$clusters | ForEach-Object {
    if ($_.TotalCPUGHz -gt 0) { $_.CPUUsagePct = [math]::Round(($_.UsedCPUGHz / $_.TotalCPUGHz) * 100, 1
) }
    if ($_.TotalMemoryGB -gt 0) { $_.MemUsagePct = [math]::Round(($_.UsedMemoryGB / $_.TotalMemoryGB) *
100, 1) }
}

# Export
$clusters | Export-Csv -Path $OutputPath -NoTypeInformation
Write-Host "Health report exported to $OutputPath"

# Disconnect
Disconnect-VIServer -Confirm:$false
```

```
# ==============================================================
# SCRIPT: Get-VcfOpsAlerts.ps1
# PURPOSE: Pull alerts from VCF Operations API
# ==============================================================

param(
    [Parameter(Mandatory=$true)]
    [string]$OpsServer,

    [Parameter(Mandatory=$true)]
    [string]$Username,

    [string]$AuthSource = "local"
)

# Ignore self-signed certs in lab
if ($PSVersionTable.PSVersion.Major -ge 6) {
    $PSDefaultParameterValues['Invoke-RestMethod:SkipCertificateCheck'] = $true
} else {
    Add-Type @"
    using System.Net;
    using System.Security.Cryptography.X509Certificates;
    public class TrustAll : ICertificatePolicy {
        public bool CheckValidationResult(ServicePoint sp, X509Certificate cert, WebRequest req, int pro
blem) { return true; }
    }
"@
    [System.Net.ServicePointManager]::CertificatePolicy = New-Object TrustAll
}

# Acquire token
$authBody = @{
    username   = $Username
    password   = Read-Host "Password" -AsSecureString | ConvertFrom-SecureString -AsPlainText
    authSource = $AuthSource
} | ConvertTo-Json

$tokenResponse = Invoke-RestMethod -Uri "https://$OpsServer/suite-api/api/auth/token/acquire" `
    -Method POST -Body $authBody -ContentType "application/json"

$token = $tokenResponse.token
$headers = @{
    "Authorization" = "OpsToken $token"
    "Accept"        = "application/json"
}
```

```
# Get alerts
$alerts = Invoke-RestMethod -Uri "https://$OpsServer/suite-api/api/alerts" `
    -Method GET -Headers $headers

# Display
$alerts.alerts | Select-Object alertId, alertLevel, status, startTimeUTC, resourceId | Format-Table -Aut
oSize

# Release token
Invoke-RestMethod -Uri "https://$OpsServer/suite-api/api/auth/token/release" `
    -Method POST -Headers $headers

Write-Host "Token released. Done."
```

DAY 28 - BUILD A "DAY-2 AUTOMATION" PACK

Time required: 3-4 hours

OBJECTIVE: Build scripts for: export health summary, list alerts, tag objects, pull inventory.

STEP-BY-STEP ACTIONS:

1. Create a folder: `vcf-ops-automation/` with subfolders: `python/`, `powershell/`, `docs/`

2. **Python Script 1: Export Health Summary**

```
#!/usr/bin/env python3
"""Export VCF Operations health summary to JSON."""

import requests
import json
import sys
from datetime import datetime

# Configuration - replace with your values or use environment variables
import os
OPS_HOST = os.environ.get("VCF_OPS_HOST", "your-ops.example.com")
OPS_USER = os.environ.get("VCF_OPS_USER", "admin")
OPS_PASS = os.environ.get("VCF_OPS_PASS", "")  # never hardcode passwords
AUTH_SOURCE = "local"

def acquire_token(host, user, password, auth_source):
    """Acquire OpsToken from VCF Operations API."""
    url = f"https://{host}/suite-api/api/auth/token/acquire"
    payload = {"username": user, "password": password, "authSource": auth_source}
    resp = requests.post(url, json=payload, verify=False)
    resp.raise_for_status()
    return resp.json()["token"]

def release_token(host, token):
    """Release OpsToken."""
    url = f"https://{host}/suite-api/api/auth/token/release"
    headers = {"Authorization": f"OpsToken {token}"}
    requests.post(url, headers=headers, verify=False)

def get_resources(host, token, resource_kind="ClusterComputeResource"):
    """Get resources of a specific kind."""
    url = f"https://{host}/suite-api/api/resources"
    headers = {"Authorization": f"OpsToken {token}", "Accept": "application/json"}
    params = {"resourceKind": resource_kind}
    resp = requests.get(url, headers=headers, params=params, verify=False)
    resp.raise_for_status()
    return resp.json()
```

```
def main():
    if not OPS_PASS:
        print("ERROR: Set VCF_OPS_PASS environment variable.")
        sys.exit(1)

    token = acquire_token(OPS_HOST, OPS_USER, OPS_PASS, AUTH_SOURCE)
    try:
        resources = get_resources(OPS_HOST, token)
        output = {
            "generated_at": datetime.utcnow().isoformat(),
            "host": OPS_HOST,
            "clusters": resources.get("resourceList", [])
        }
        filename = f"health_summary_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
        with open(filename, "w") as f:
            json.dump(output, f, indent=2)
        print(f"Health summary exported to {filename}")
    finally:
        release_token(OPS_HOST, token)

if __name__ == "__main__":
    main()
```

3. **Python Script 2: List Active Alerts**

```
#!/usr/bin/env python3
"""List active alerts from VCF Operations API."""

import requests
import json
import os
import sys

OPS_HOST = os.environ.get("VCF_OPS_HOST", "your-ops.example.com")
OPS_USER = os.environ.get("VCF_OPS_USER", "admin")
OPS_PASS = os.environ.get("VCF_OPS_PASS", "")
AUTH_SOURCE = "local"

def acquire_token(host, user, password, auth_source):
    url = f"https://{host}/suite-api/api/auth/token/acquire"
    payload = {"username": user, "password": password, "authSource": auth_source}
    resp = requests.post(url, json=payload, verify=False)
    resp.raise_for_status()
    return resp.json()["token"]

def release_token(host, token):
    url = f"https://{host}/suite-api/api/auth/token/release"
    headers = {"Authorization": f"OpsToken {token}"}
    requests.post(url, headers=headers, verify=False)

def get_alerts(host, token, status="ACTIVE"):
    url = f"https://{host}/suite-api/api/alerts"
    headers = {"Authorization": f"OpsToken {token}", "Accept": "application/json"}
    params = {"status": status}
    resp = requests.get(url, headers=headers, params=params, verify=False)
    resp.raise_for_status()
    return resp.json()

def main():
    if not OPS_PASS:
        print("ERROR: Set VCF_OPS_PASS environment variable.")
        sys.exit(1)

    token = acquire_token(OPS_HOST, OPS_USER, OPS_PASS, AUTH_SOURCE)
    try:
```

```
        alerts = get_alerts(OPS_HOST, token)
        alert_list = alerts.get("alerts", [])

        print(f"\n{'='*80}")
        print(f"ACTIVE ALERTS: {len(alert_list)}")
        print(f"{'='*80}")

        for alert in alert_list:
            print(f"\n  Severity: {alert.get('alertLevel', 'UNKNOWN')}")
            print(f"  Alert:    {alert.get('alertDefinitionName', 'N/A')}")
            print(f"  Object:   {alert.get('resourceId', 'N/A')}")
            print(f"  Status:   {alert.get('status', 'N/A')}")
            print(f"  Started:  {alert.get('startTimeUTC', 'N/A')}")
            print(f"  ---")
    finally:
        release_token(OPS_HOST, token)

if __name__ == "__main__":
    main()
```

4. **Test each script** (set environment variables first):

```
   export VCF_OPS_HOST="your-ops.example.com"
   export VCF_OPS_USER="admin"
   export VCF_OPS_PASS="your-password"
   python3 python/export_health_summary.py
   python3 python/list_alerts.py
```

DELIVERABLE: Working automation pack with at least 4 scripts (2 Python, 2 PowerShell from Day 27).

DAY 29 - CAPSTONE: "OPS COMMAND CENTER"

Time required: 4-5 hours

OBJECTIVE: Combine everything into a cohesive operations package.

STEP-BY-STEP ACTIONS:

1. **Assemble your Ops Command Center package:**

```
YOUR OPS COMMAND CENTER - FINAL PACKAGE
========================================

DASHBOARDS (in VCF Operations):
  [x] NOC - Real-Time Status (Day 11)
  [x] OPS - Daily Operations View (Day 11)
  [x] EXEC - Weekly Infrastructure Summary (Day 11)
  [x] NOC-Logs - Error Trends (Day 18)
  [x] SEC-Logs - Auth Failures (Day 18)

RUNBOOKS (documents):
  [x] Triage Runbook v1 (Day 3)
  [x] Alert Taxonomy P1-P4 (Day 8)
  [x] Log-Based Alert Playbook (Day 19)
  [x] RCA Template (Day 13)
  [x] Weekly Capacity Report Template (Day 9)

AUTOMATION (scripts):
  [x] Get-VcfOpsHealth.ps1 (Day 27)
  [x] Get-VcfOpsAlerts.ps1 (Day 27)
  [x] export_health_summary.py (Day 28)
  [x] list_alerts.py (Day 28)
```

```
REFERENCE DOCS:
  [x] VCF Ops Navigation Map (Day 1)
  [x] Object Relationship Cheat Sheet (Day 2)
  [x] Drift Watchlist (Day 10)
  [x] API Quickstart (Day 22)
  [x] Auth Cheat Sheet (Day 23)
  [x] Postman Collection (Day 24)
  [x] Automation Path Chooser (Day 25)
  [x] Log Source Inventory Checklist (Day 15)
  [x] Top 10 Log Dashboards to Pin (Day 17)
```

2. **Run through each item and verify it works:**

• Open each dashboard - does it load? Does data display?

• Read each runbook - does it make sense? Any gaps?

• Run each script - does it execute without errors?

• Check each reference doc - is it accurate and complete?

3. **Create your "Command Center" landing dashboard:**

• In VCF Operations, create one more dashboard: `OPS COMMAND CENTER - HOME`

• This is your single entry point. Widgets:

• **Row 1:** Overall Health (all domains) | Active Alert Count | Capacity Runway (days)

• **Row 2:** Navigation Links (text widget with links to your other dashboards)

• **Row 3:** Recent Incidents (text widget you update manually)

DAY 30 - FINAL BOSS: MOCK INTERVIEW + LIVE DEMO

Time required: 3-4 hours (45 min rapid fire + 15 min dashboard walkthrough + 10 min automation demo + prep)

PREPARATION (1 hour before):

• Open all your dashboards in browser tabs

• Have your scripts ready to run in a terminal

• Have your runbooks/cheat sheets printed or on a second screen

• Dress like it is a real interview (yes, even alone - it builds the mindset)

PART 1: RAPID FIRE QUESTIONS (45 minutes)

Have someone ask you these (or record yourself). Speak your answers out loud.

Q1: "What is VCF Operations?"

SAY THIS: "VCF Operations is the unified management console for VMware Cloud Foundation 9. In VCF 9.0, the SDDC Manager UI was deprecated and its workflows moved into VCF Operations, making it the true single pane of glass. It collects metrics, logs, and configuration data from all VCF components - vCenter, ESXi hosts, NSX, vSAN - and provides health monitoring through VCF Health and Diagnostic Findings, performance analytics, capacity planning with cost visualization, compliance checking under Security, log analysis through Analyze, fleet management including lifecycle, certificates, and passwords, and custom dashboards and reports. The left nav has Infrastructure Operations with direct items like Diagnostic Findings, VCF Health, Dashboards & Reports, Alerts, Troubleshooting Workbench, Analyze, Storage Operations, and Network Operations, plus expandable sections for Workload Operations, Fleet Management, Capacity, Security, Administration, and Developer Center."

Q2: "Walk me through your first 15 minutes when you get paged for a production issue."

SAY THIS: "I follow a structured triage runbook. Minutes 0-2: I check VCF Health and Diagnostic Findings under Infrastructure Operations to assess scope - Diagnostic Findings automatically correlates issues against 107 known-issue signatures from Broadcom Support, so it often identifies the problem before I start digging. Minutes 2-5: I go to Infrastructure Operations > Alerts, filter to Active Critical and Immediate, sort newest first. I record each alert's name, affected object, and recommendation. Minutes 5-8: I click into the most critical affected object and go to the Relationships tab to determine blast radius - how many VMs, users, or services depend on this object. If I need deeper investigation, I open a Troubleshooting Workbench session to get the Object Relationship Graph. Minutes 8-12: I check the Metrics tab for the anomalous metric and if I need log evidence, I use Infrastructure Operations > Analyze to search log data. Minutes 12-15: I document my findings, determine next action - fix it, escalate, or monitor - and update the incident ticket."

Q3: "Walk me through a dashboard you built that reduced MTTR."

SAY THIS: "I built a Daily Health Overview dashboard with four widgets: cluster health scoreboard, top 10 VMs by CPU usage, active critical alerts list, and cluster capacity remaining. The key feature is widget interactions - when I click a cluster in the health scoreboard, all other widgets filter to that cluster. Before this dashboard, triage meant clicking through multiple screens to correlate alerts with affected objects. Now I can go from 'cluster X is unhealthy' to 'these specific VMs are causing the problem' in one click. This cut our initial triage time from about 15 minutes to under 5 minutes."

Q4: "How do you decide what goes on a NOC dashboard versus an executive dashboard?"

SAY THIS: "They serve different audiences with different questions. The NOC dashboard answers 'what is broken right now' - it shows real-time health status, active critical alerts, service availability, and auto-refreshes every 60 seconds. The NOC operator needs to see problems instantly and begin triage. The executive dashboard answers 'are we healthy and do we need to spend money' - it shows aggregate health by workload domain as green/yellow/red, capacity runway in weeks, top risks, and efficiency metrics. No individual VM data, no raw alert counts - just strategic indicators. I also build a VMware Ops team dashboard that sits between the two - it shows daily health trends, capacity planning data, compliance scores, and recent configuration changes."

Q5: "How do you handle alert noise?"

SAY THIS: "I take a systematic approach. First, I audit the alert landscape by counting active alerts by severity over the last 7 days. If Critical plus Immediate exceeds 50, there is a noise problem. I sort alerts by frequency to find the top offenders - alerts that fire and clear repeatedly. For each noisy alert, I ask: does this require a human to take action? If not, I either raise the threshold, extend the trigger duration, or disable it. I classify all alerts into P1 through P4 with clear definitions. P1 is service-impacting with a 15-minute response target. P2 is significant degradation with a 1-hour target. P3 is non-urgent with a business-day target. P4 is informational reviewed weekly. Each level has a defined owner and escalation path. The goal is: every alert that fires should result in a human taking a specific action."

Q6: "How does API authentication work in VCF Operations?"

SAY THIS: "VCF Operations uses token-based authentication. You POST credentials to the token acquire endpoint at /suite-api/api/auth/token/acquire with a JSON body containing username, password, and authSource. For local accounts, authSource is 'local'. For SSO accounts, it is the identity source name like 'vsphere.local'. The response contains a token that is valid for approximately 6 hours. For all subsequent API calls, you include the header 'Authorization: OpsToken' followed by a space and the token value. When you are done, you should release the token by POSTing to the token release endpoint. The Swagger UI is at /suite-api/doc/swagger-ui.html where you can explore and test all available endpoints."

Q7: "What is configuration drift and why do you care?"

SAY THIS: "Configuration drift is when a system's actual configuration deviates from its approved baseline without going through change management. I care about it for three reasons. Security: an enabled SSH service or a weakened password policy is an attack surface. Stability: NTP skew can break vSAN, certificate validation, and log correlation. Compliance: auditors require proof that configurations match approved baselines. I maintain a drift

watchlist of the top 10 settings I track: SSH service status, NTP configuration, syslog forwarding, lockdown mode, SSO password policy, firewall rules, VM hardware version, datastore permissions, distributed switch security settings, and certificate validity. Any change to these without a change ticket triggers an investigation."

Q8: "How do you use logs differently for troubleshooting versus auditing?"

SAY THIS: "Troubleshooting logs are reactive and real-time. I use them to find root cause of current issues - searching vmkernel.log for storage errors, vpxd.log for vCenter service failures, NSX syslog for network issues. I need recent detailed data, typically the last 7 to 30 days. The question I am answering is 'why did this break?' Audit logs are proactive and historical. I use them to track who did what, when - vCenter event logs for VM operations, ESXi shell access logs, NSX audit logs for firewall rule changes, SSO login logs. I need longer retention, 90 days to a year. The question I am answering is 'who changed this and were they authorized?' In Operations-Logs, I build separate dashboards for each: a troubleshooting board with error trends and service health, and a security board with authentication failures, unauthorized access, and configuration changes."

Q9: "How do content packs help in Operations-Logs?"

SAY THIS: "Content packs are pre-built bundles of dashboards, alerts, and saved queries designed for specific log sources. For example, the vSphere content pack includes dashboards for ESXi host errors, vCenter events, and authentication tracking, plus alert definitions for common failure patterns, plus saved queries for frequent troubleshooting scenarios. They accelerate time-to-value because instead of building every dashboard and alert from scratch, I install the content pack and immediately have a functional monitoring baseline. I then customize on top of that baseline for my specific environment. It is the same principle as using a template instead of starting from a blank page."

Q10: "When do you automate and when do you not?"

SAY THIS: "I automate when three conditions are met: the task is repeatable, meaning I do it more than once a week. The task is deterministic, meaning the same inputs always produce the same correct outputs. And the blast radius of a mistake is manageable, meaning if the script fails, it does not take down production. For example, exporting a daily health report is a perfect automation candidate - repeatable, deterministic, read-only. Provisioning a new workload domain is automatable but requires guardrails - validation checks, dry-run mode, and human approval before execution. I do not automate tasks that require judgment calls, like deciding whether to evacuate a host during a performance event. I also follow the principle of idempotency - my scripts should be safe to run twice without causing damage."

PART 2: DASHBOARD WALKTHROUGH (15 minutes)

Open your Ops Command Center - HOME dashboard and walk through it:

SAY THIS: "This is my Ops Command Center. It is the single entry point I use every morning.

[Point to Row 1] At the top I have three key indicators: overall health across all workload domains, the count of active alerts broken down by severity, and the capacity runway showing the most constrained cluster and how many days until it reaches capacity.

[Point to Row 2] In the middle I have my navigation hub linking to three audience-specific dashboards: the NOC real-time status board for incident response, the Ops daily operations view for my team's day-to-day work, and the Executive weekly summary for leadership briefings.

Let me drill into the NOC dashboard. [Click into NOC dashboard] This auto-refreshes every 60 seconds. The heat map shows cluster health at a glance - all green means I can move on. If I see yellow or red, I click it and the alert list below filters to show only alerts for that cluster. Let me demonstrate. [Click a cluster, show the filtering]

Now let me show the log side. [Open the Auth Failures dashboard] This shows failed authentication attempts across all components over the last 7 days. I can see patterns - if one username has 50 failures in 10 minutes, that is a brute force attempt. [Point to data] This dashboard helped me identify an unauthorized access attempt last week and respond within minutes instead of finding out during an audit."

PART 3: AUTOMATION DEMO (10 minutes)

Open your terminal and run your scripts:

SAY THIS: "I have built a day-2 automation pack with Python and PowerShell scripts. Let me demonstrate.

[Open terminal] First, I will run the health summary export. This script authenticates to the VCF Operations API using token-based auth, pulls cluster resource data, and exports to JSON.

[Run the script] python3 export_health_summary.py

[Show the output file] Here is the exported JSON with cluster data. This runs as a scheduled task every morning at 6 AM and feeds into our daily standup.

[Run the alerts script] python3 list_alerts.py

[Show the output] This pulls all active alerts and displays them in a readable format. In production, I have this sending to a Slack channel so the team sees new critical alerts immediately.

The key principles in my automation: credentials are never hardcoded - they come from environment variables. Tokens are always released after use. All API calls use error handling. Scripts are idempotent - safe to run multiple times. And I start with read-only operations before automating any write operations."

SCORING RUBRIC (Final Assessment):

```
SIGNAL OVER NOISE: Do you choose the right metrics and alerts?  /10
ORDER OF OPERATIONS: Do you triage in a sane sequence?           /10
STORYTELLING: Do you communicate impact and prevention?          /10
AUTOMATION MATURITY: Safe, tested, minimal blast radius?         /10
DASHBOARD DESIGN: Decision-making dashboards, not wallpaper?     /10
API COMFORT: Can you explain and demonstrate token auth?         /10
LOG MASTERY: Can you pivot from symptoms to log proof?           /10
OVERALL CONFIDENCE: Can you present without hesitation?          /10

TOTAL: /80
70+: Ready for interviews
55-69: Need more reps on weak areas
Below 55: Repeat Week 3-4 before interviewing
```

VCF OPERATIONS INTERVIEW PREP PACK - COMPLETE

Exact Answers | Scoring Rubrics | Whiteboard Prompts

A. YOUR 60-SECOND PITCH

When the interviewer says "Tell me about yourself" or "Walk me through your background":

SAY THIS (practice until it is natural, under 60 seconds):

"I am an infrastructure operations engineer focused on VMware Cloud Foundation environments. My core strengths are in three areas. First, monitoring and operations: I build decision-making dashboards in VCF Operations that give NOC teams, ops teams, and executives the right view for their role. I have reduced triage time by implementing widget interactions and structured runbooks. Second, performance and capacity management: I run weekly capacity reviews, track configuration drift, and proactively identify risks before they become outages. Third, automation: I use the VCF Operations API, Python, and PowerCLI to automate repeatable tasks like health exports, alert reporting, and compliance checking. I follow a principle of automating what is repeatable, deterministic, and safe before touching anything that requires judgment. I am looking for a role where I can bring this operational discipline to a VCF environment at scale."

B. COMPLETE QUESTION BANK WITH EXACT ANSWERS

DASHBOARDS & OPERATIONS (20 Questions)

Q1: "Walk me through a dashboard you built that reduced MTTR."

SAY THIS: "I built a Daily Health Overview dashboard with four widgets: a cluster health scoreboard showing green, yellow, or red status for each cluster, a Top-10 VMs by CPU chart to spot resource consumers, an active critical alerts list filtered to Critical and Immediate severity only, and a cluster capacity remaining scoreboard. The key design decision was widget interactions. When I click a specific cluster in the health scoreboard, the VM list and alert list automatically filter to show only data for that cluster. Before this dashboard, triage meant navigating through Inventory, then Alerts, then Metrics across multiple screens. After, we could go from 'cluster X is degraded' to 'these are the specific VMs and alerts' in a single click. This cut initial triage from about 15 minutes to under 5, which directly reduced our MTTR on cluster-level issues."

Q2: "How do you decide what goes on a NOC dashboard vs an exec dashboard?"

SAY THIS: "They answer different questions for different audiences. The NOC dashboard answers 'what is broken right now.' It needs real-time health status with auto-refresh every 60 seconds, active Critical and Immediate alerts sorted newest first, service availability indicators, and host status. The NOC operator needs to see a problem and start triage in seconds, not minutes. The executive dashboard answers 'are we healthy and do we need to invest.' It shows aggregate health per workload domain as green, yellow, or red badges, capacity runway in weeks or months, a top-3 risk summary, and resource efficiency percentages. No individual VM details. No raw alert lists. No technical metric names. I also maintain an Ops Team dashboard in between that covers daily health trends, capacity planning detail, compliance scores, and configuration change tracking."

Q3: "How do you avoid dashboard sprawl?"

SAY THIS: "Five practices. First, naming convention: every dashboard starts with a prefix indicating its audience - NOC, OPS, EXEC, SEC - so they are easy to find and categorize. Second, I use Favorites to pin the dashboards I actually use daily, and review Recents weekly to see if I am ignoring any. Third, I clone prebuilt dashboards and customize them instead of building from scratch, which prevents duplicate effort. Fourth, I run a monthly review using the Manage tab under Dashboards & Reports to identify and delete unused dashboards - if no one opened it in 30 days, it gets archived or deleted. Fifth, I design dashboards with widget interactions so one dashboard can serve multiple drill-down paths instead of needing five separate static dashboards."

Q4: "What is your process for turning noisy alerts into a signal?"

SAY THIS: "I start with an audit. I count active alerts by severity over the last 7 days. If Critical plus Immediate exceeds 50 for our environment size, we have a noise problem. I sort by frequency to find the top offenders - alerts that fire and auto-cancel repeatedly. For each noisy alert, I ask three questions: Does this require a human to take

a specific action? If the answer is always 'ignore it,' the alert needs to change. Can I raise the threshold? For example, a CPU alert at 80% might fire constantly but never be actionable, while 95% sustained for 10 minutes actually means something. Can I extend the trigger duration? A 1-minute spike is noise; a 15-minute sustained condition is signal. Then I classify all alerts into P1 through P4 with clear ownership and MTTR targets. P1 pages the on-call engineer with a 15-minute response. P2 notifies the team lead with a 1-hour response. P3 goes into the daily standup queue. P4 is reviewed weekly. Every alert that fires should have a corresponding action."

Q5: "What are your top 5 daily checks in VCF Operations?"

SAY THIS: "Every morning I follow the same sequence. First, I check Infrastructure Operations > VCF Health for full-stack health across all workload domains. Green means move on, anything else means investigate. I also check Diagnostic Findings which automatically scans for 107 known-issue signatures from Broadcom Support. Second, I check Infrastructure Operations > Alerts for new Critical or Immediate alerts that fired overnight. If so, I check if they were already acknowledged or need attention. Third, capacity check: I go to Capacity > Capacity Optimization and glance at capacity remaining percentages for production clusters. Anything below 30% gets flagged for the weekly capacity review. Fourth, I check Security > Compliance for new drift findings and Fleet Management > Configuration Management for any scheduled drift detection results. Fifth, I check Infrastructure Operations > Analyze to verify log ingestion is flowing from all sources. If a source stopped sending logs overnight, I need to investigate before I lose forensic visibility."

Q6: "Describe widget interactions in VCF Operations dashboards."

SAY THIS: "Widget interactions allow one widget to drive the content of another widget on the same dashboard. When I configure an interaction, I define a source widget and a target widget. When I select an object in the source widget, the target widget updates to show data only for that selected object. For example, I have a cluster health scoreboard as my source and a VM list and alert list as targets. When I click Cluster-Prod-01 in the scoreboard, the VM list shows only VMs in that cluster and the alert list shows only alerts for that cluster. I can chain multiple targets from one source, so one click can update three or four other widgets simultaneously. This is what makes dashboards into investigation tools instead of static wall displays."

Q7: "How do you manage dashboards across teams?"

SAY THIS: "I use the Manage tab under Infrastructure Operations > Dashboards & Reports to control sharing and permissions. Each dashboard has a share setting that determines who can see it. I share NOC dashboards with the NOC operator group, Ops dashboards with my operations team, and Exec dashboards with the management group. I also use the clone feature when teams need similar dashboards with different scope - I clone the template and modify the object scope for each team. For governance, I own all dashboards in our environment and run a monthly review to clean up unused ones, update out-of-date ones, and add new ones based on team feedback."

Q8: "What makes a good operations dashboard?"

SAY THIS: "A good operations dashboard answers a specific question for a specific audience and drives a specific action. It is not decoration. Four criteria. First, it has a clear purpose stated in the title - 'NOC Real-Time Status' not 'Dashboard 7.' Second, it uses the right widgets for the data type - scoreboards for status, Top-N for comparisons, time-series charts for trends, tables for detail. Third, it has widget interactions so the operator can drill down without leaving the page. Fourth, it is actionable - every piece of data on the dashboard should either confirm 'everything is fine' or trigger a specific next step. If a widget is on the dashboard and no one ever looks at it or acts on it, it should be removed."

Q9: "How do you validate that a dashboard is useful?"

SAY THIS: "I watch how people actually use it. After deploying a new dashboard, I check three things over the first two weeks. First, is it being opened? If it has zero views, it is not meeting a need. Second, are people interacting with it - clicking widgets, using filters - or just glancing at it? Interaction means they are using it for investigation, not just decoration. Third, I ask the users directly: 'When you opened this dashboard today, did it answer your question? What was missing?' I have removed widgets that seemed important to me but users told me they never looked at. I have added widgets that users requested because their actual workflow needed information I did not

anticipate."

Q10: "How do you build a capacity story for leadership?"

SAY THIS: "I present capacity in business terms, not technical metrics. Instead of saying 'Cluster 3 is at 78% CPU,' I say 'Cluster 3 can absorb approximately 40 more VMs at current average VM size before we need additional hardware, and at current growth rate that is about 90 days.' My weekly capacity report has five sections: executive summary with the headline and recommended action, capacity status by cluster in a table with percentages and days to exhaustion, top capacity risks ranked by urgency, reclaimable capacity from oversized VMs and old snapshots that could defer hardware purchases, and specific recommendations with owners and dates. I always include a trend - is capacity usage growing faster, slower, or steady compared to last month."

Q11: "Tell me about a time you identified and resolved a performance issue."

SAY THIS: "On a Tuesday afternoon, production users reported application slowness. I opened my NOC dashboard and saw Cluster-Prod-01 health was Yellow with CPU contention. I clicked the cluster in the health scoreboard - the widget interaction filtered my alert list to show a Critical alert for sustained CPU usage above 92%. I went to the Metrics tab and confirmed CPU had spiked at 2:32 PM and remained elevated. In the Top-N widget, I identified three VMs consuming 45% of cluster CPU. These were batch processing VMs that had started unscheduled data jobs. I vMotioned eight workload VMs to Cluster-Prod-02 to relieve immediate pressure, then worked with the application team to move the batch VMs to a dedicated resource pool with CPU limits. I documented the RCA, created an alert threshold at 85% for earlier warning, and set the batch jobs to run only during off-peak hours. Total impact was 43 minutes of application latency affecting approximately 200 users."

Q12: "How do you write a root cause analysis?"

SAY THIS: "My RCA follows a standard template with seven sections. Executive summary: two to three sentences covering what happened, who was affected, and how it was resolved. Timeline: a minute-by-minute table from first symptom to resolution. Root cause: the specific technical cause, stated precisely. Impact: number of users, services, and duration. Resolution: what was done immediately and what technical changes were made. Prevention using the 5 Whys method: I trace the chain of causation back to the systemic issue and define corrective actions with owners and due dates. Lessons learned: what worked well in the response, what could be improved, and what should be automated. The most important part is the prevention section. If the RCA does not result in changes that prevent recurrence, it was just documentation, not improvement."

Q13: "What is configuration drift and how do you handle it?"

SAY THIS: "Configuration drift is when a system's actual configuration deviates from its approved baseline without going through change management. In VCF 9.0, I handle it with two tools in two places. First, Security > Compliance checks against industry benchmarks and hardening guides like CIS and the vSphere Security Configuration Guide. It discovers non-compliant objects and provides remediation guidance. Second, Fleet Management > Configuration Management does scheduled drift detection against my own baselines for vCenter and cluster objects. It integrates with Git for template versioning so I can track changes over time, and it generates PDF drift reports. I maintain a top-10 drift watchlist of the settings that cause the most damage when they change - SSH status, NTP, syslog forwarding, lockdown mode, password policies, firewall rules, VM hardware version, datastore permissions, DVS security settings, and certificate validity. I also use Security > Audit Events to investigate who changed what and when. If the same drift keeps recurring, I automate the remediation or fix the root cause in the provisioning process."

Q14: "How do you present a weekly ops briefing?"

SAY THIS: "My weekly briefing is exactly five minutes, structured into five one-minute sections. Minute one: overall health status across all domains, active alert counts, any P1 incidents that occurred. Minute two: capacity status by cluster, trending data, any clusters approaching thresholds. Minute three: performance highlights and any incidents with brief RCA summaries. Minute four: compliance status, drift findings, certificate expirations, security observations. Minute five: action items with owners and dates, and a call for questions. I present from my dashboards, not from slides. The dashboards are live data, which builds credibility and allows me to drill into any

question on the spot."

Q15: "What is the difference between VCF Operations and SDDC Manager?"

SAY THIS: "In VCF 9.0, they converged. The SDDC Manager UI is deprecated - its workflows for lifecycle management, fleet management, certificate management, password management, and configuration management all moved into VCF Operations. VCF Operations is now the single unified console for both operations and lifecycle. Under Fleet Management in the left nav, I find Lifecycle Management for upgrades and patching, Certificate Management for cert renewal and replacement, Password Management for credential rotation, and Configuration Management for drift detection with Git integration. Under Infrastructure Operations, I find the monitoring side - VCF Health, Diagnostic Findings with 107 known-issue signatures, Alerts, Dashboards & Reports, Troubleshooting Workbench with Object Relationship Graphs, and log analysis via Analyze. Some SDDC Manager deployment tasks also moved to the vSphere Client. The bottom line: VCF Operations IS the operational control plane now, not just the monitoring layer."

Q16: "How do you handle an alert that keeps firing and clearing repeatedly?"

SAY THIS: "A flapping alert is a sign that the threshold is too close to normal operating range. I take four steps. First, I look at the metric history for the affected object to understand its normal range and variance. Second, I check whether the alert threshold makes sense for this specific object - a host running a database may normally sit at 85% memory, which is fine. Third, I either raise the threshold to above the normal peak, extend the trigger duration so short spikes do not fire the alert, or create a custom alert definition for objects with known higher baselines. Fourth, if the flapping indicates an actual intermittent problem, like a host dropping off the network and reconnecting, I investigate the underlying issue instead of just silencing the alert."

Q17: "What metrics tell you a cluster is in trouble?"

SAY THIS: "I look at four metrics in order. First, CPU Ready time, not just CPU usage. High CPU ready means VMs are waiting for CPU cycles and experiencing contention - this is the metric users feel. Second, memory ballooning and swapping. If the VMkernel is reclaiming memory via balloon driver or swapping to disk, performance is already degraded. Third, disk latency. Read and write latency above 20 milliseconds sustained indicates storage contention or a failing storage path. Fourth, network dropped packets. Any non-zero drop count means something is saturated or misconfigured. I weight these differently: CPU ready and memory swapping are immediate user-impacting, disk latency could be a hardware issue developing, and network drops could indicate a configuration problem."

Q18: "How do you identify oversized VMs?"

SAY THIS: "In VCF Operations, I look at the capacity analytics for reclaimable resources. Specifically, I compare allocated vCPU versus actual CPU usage and allocated memory versus actual memory active. A VM with 8 vCPU allocated but averaging 1.5 vCPU used is oversized. I also look at the CPU ready time - an oversized VM with too many vCPUs can actually hurt performance because the scheduler has to find that many free physical cores simultaneously. My right-sizing recommendation follows a rule: set allocation to the 95th percentile of actual usage plus a 20% buffer. I present right-sizing as a capacity reclamation opportunity, not a punishment. I show leadership how much capacity we can free up across the fleet by right-sizing the top 20 most oversized VMs."

Q19: "What would you automate first in a new VCF Operations environment?"

SAY THIS: "I follow a safe-to-impactful progression. First, I automate read-only health exports - a daily script that pulls cluster health, alert counts, and capacity metrics and saves them to a file or sends them to a channel. Zero risk, immediate value. Second, I automate the weekly capacity report so it is generated automatically instead of manually pulled from the UI. Third, I automate snapshot cleanup reporting - a script that finds all snapshots older than 7 days and generates a report for review, but does not delete anything without approval. Fourth, I automate certificate expiration monitoring - a script that checks all certificate dates and alerts if anything expires in under 30 days. These four give me daily health visibility, weekly capacity planning, storage hygiene, and security compliance, all without any write operations that could cause damage."

Q20: "Describe your ideal VCF monitoring setup."

SAY THIS: "In VCF 9.0, it is all one unified console. Under Infrastructure Operations, I use VCF Health and Diagnostic Findings for stack-wide health monitoring with 107 known-issue signatures. Dashboards & Reports gives me customizable dashboards with widget interactions for drill-down. Alerts handles metric-based alerting classified P1-P4. Storage Operations and Network Operations give me specialized views for vSAN and NSX. Analyze gives me integrated log analysis with saved queries, extracted fields, and log-based alerts - this requires the VCF Operations for Logs component. Security > Compliance checks against CIS benchmarks, and Fleet Management > Configuration Management handles drift detection with Git-backed template versioning. Under Capacity, I get cost visualization, capacity optimization, and what-if scenarios. Then layer three is automation via the Developer Center REST APIs and PowerCLI: scripts that export health data daily, generate capacity reports weekly, and monitor certificates via Fleet Management > Certificate Management. Everything is in one console now, not three separate tools."

API & AUTOMATION (20 Questions)

Q21: "Show me how you would find the API documentation for VCF Operations."

SAY THIS: "The VCF Operations API documentation is hosted as Swagger UI directly on the appliance. I navigate to https://<ops-fqdn>/suite-api/doc/swagger-ui.html in a browser. This shows every available API endpoint organized by category - Authentication, Alerts, Resources, Stats, Dashboards, Reports, and more. Each endpoint shows the HTTP method, URL path, parameters, request body schema, and response schema. I can also use the 'Try it out' feature to execute API calls directly from the browser. For offline reference, I can download the OpenAPI spec file and import it into Postman to generate a full collection."

Q22: "How does token auth work in VCF Operations API?"

SAY THIS: "I POST a JSON body with username, password, and authSource to /suite-api/api/auth/token/acquire. For local accounts, authSource is 'local.' For SSO accounts, it is the identity source name like 'vsphere.local.' The API returns a JSON response containing a token string and its validity period, typically 6 hours. For every subsequent API call, I include the header Authorization: OpsToken followed by a space and the token value. When I am done with my session, I POST to /suite-api/api/auth/token/release with the same Authorization header to invalidate the token. Common mistakes are forgetting the 'OpsToken' prefix before the token, letting the token expire without acquiring a new one, using the wrong authSource value, and not setting the Accept header to application/json which can result in XML responses instead of JSON."

Q23: "Why does OpenAPI matter for VCF automation?"

SAY THIS: "OpenAPI specifications are machine-readable descriptions of the entire API surface. Three practical benefits. First, I can import the spec into Postman and it automatically generates a complete collection with every endpoint, parameter, and example body pre-configured. This skips hours of manual collection building. Second, I can use code generation tools like openapi-generator to produce client libraries in any language - Python, Java, Go, whatever the team uses. The generated code handles serialization, error types, and parameter validation. Third, the spec serves as always-accurate documentation because it is generated from the actual API code. If an endpoint changes in a new version, the spec reflects it, so my generated clients stay in sync."

Q24: "Where do SDKs fit in VCF automation?"

SAY THIS: "Broadcom publishes a Unified VCF SDK for Python and Java. The SDK provides structured, object-oriented wrappers around the REST APIs, which is easier to work with than raw HTTP calls. I use the SDK when building reusable automation tools or integrations where I need error handling, type safety, and maintainability. For quick one-off API calls, I use Postman or curl. For operational scripting by VMware admins who know PowerShell, I use VCF PowerCLI. For generating client code in languages the SDK does not cover, I use the OpenAPI specs with code generators. The decision comes down to: who will maintain this code and what language are they comfortable with."

Q25: "Explain VCF PowerCLI in VCF 9."

SAY THIS: "VCF PowerCLI is the renamed and updated PowerShell module for managing VCF services. In VCF 9, it is the operational scripting tool for administrators who work in PowerShell. I install it with Install-Module VMware.PowerCLI, connect to vCenter with Connect-VIServer, and then have access to cmdlets for managing VMs, hosts, clusters, networking, and storage. For VCF-specific operations like managing workload domains, host commissioning, and lifecycle tasks, I use the SDDC Manager API endpoints. PowerCLI is my choice for operational tasks that admins run interactively or on a schedule - things like health checks, inventory exports, snapshot cleanup, and configuration audits."

Q26: "How do you keep credentials out of automation scripts?"

SAY THIS: "Three methods depending on the context. For scripts running on my workstation, I use environment variables. The script reads the password from an environment variable that I set in my shell session, never in the script file. For scheduled tasks, I use a credential store or secrets manager. PowerCLI has a credential store feature, and for Python I use either a secrets manager API or encrypted credential files. For CI/CD pipelines, I use pipeline secrets or vault integration - the secret is injected at runtime and never appears in the code repository. The principle is: credentials should never exist in source code, configuration files checked into version control, or command-line arguments visible in process listings."

Q27: "What is idempotency and why does it matter for automation?"

SAY THIS: "Idempotency means running the same script twice produces the same result as running it once. It matters because in operations, scripts fail midway, get re-run, get scheduled to overlap, or get triggered by multiple events. If my script creates a resource without checking if it already exists, running it twice creates duplicates. If my script deletes and recreates, running it during a partial failure could destroy data. An idempotent script checks state before acting. For example, before creating an alert definition, it checks if one with that name already exists. Before setting a configuration, it reads the current value and only changes it if different. Before adding a host to inventory, it checks if the host is already present. This is what makes automation safe to run in real environments."

Q28: "Walk me through building a Postman collection for VCF Operations."

SAY THIS: "I start by creating a Postman environment with variables: ops_host for the server FQDN, ops_token which starts empty, ops_user and ops_pass for credentials. Then I create the collection with folders matching the API categories: Auth, Alerts, Resources, Stats, Dashboards. The first request is Acquire Token - a POST to the token endpoint using environment variables for credentials. In the Tests tab, I add JavaScript that extracts the token from the response and saves it to the ops_token environment variable. Every subsequent request uses the Authorization header with value 'OpsToken' followed by the ops_token variable. This way, I run the auth request once and all other requests automatically use the valid token. If I have the OpenAPI spec file, I can skip manual creation entirely by importing the spec, which auto-generates the entire collection."

Q29: "How do you test automation scripts safely before running in production?"

SAY THIS: "Four-stage approach. Stage one: I test in a lab or dev environment that mirrors production but has no production workloads. Stage two: I run the script in read-only mode first. My scripts have a dry-run flag that logs what they would do without actually doing it. Stage three: I run against a single object in production - one host, one cluster, one VM - and verify the result before expanding scope. Stage four: I run against the full target set with monitoring. For write operations, I always include a confirmation prompt unless the script is running unattended on a schedule, in which case the validation is built into the pre-checks. I also keep logs of every action the script takes so I can audit and roll back if needed."

Q30: "What REST API calls would you make to pull a health summary?"

SAY THIS: "Three calls. First, POST to /suite-api/api/auth/token/acquire to get my authentication token. Second, GET to /suite-api/api/resources with a resourceKind parameter set to ClusterComputeResource to get all cluster objects and their current health status. I can add a pageSize parameter to handle pagination in large environments. Third, for each cluster, I can optionally GET /suite-api/api/resources/{resourceId}/stats with specific metric keys to pull CPU usage, memory usage, and capacity remaining. Finally, I POST to

/suite-api/api/auth/token/release to clean up my token. The result is a complete health picture that I can serialize to JSON, format into a report, or push to a monitoring channel."

LOGS (20 Questions)

Q31: "How do you use logs differently for auditing versus troubleshooting?"

SAY THIS: "Troubleshooting is reactive and time-bounded. I search for specific error messages, stack traces, or service status changes around the time an issue was reported. I need recent, detailed data, typically 7 to 30 days. The key log sources are vmkernel.log for host issues, vpxd.log for vCenter problems, and NSX syslog for network events. Auditing is proactive and broad. I look for patterns over longer time periods - who logged in, what changed, when, and was it authorized. I need 90 days to a year of retention. The key sources are vCenter events and tasks, ESXi shell.log and auth.log, NSX audit logs, and SSO authentication logs. In Operations-Logs, I build separate dashboards for each purpose: a troubleshooting board focused on error rates and service health, and a security audit board focused on authentication events, privilege escalation, and configuration changes."

Q32: "How do content packs help in Operations-Logs?"

SAY THIS: "Content packs are pre-built bundles of dashboards, alert definitions, and saved queries for specific log sources. For example, the vSphere content pack includes dashboards showing ESXi host error trends, vCenter service health, and authentication tracking, plus alert definitions for patterns like repeated login failures or service crashes, plus saved queries for common troubleshooting scenarios. The value is speed to operational readiness. Instead of spending days building dashboards and writing queries from scratch, I install the relevant content packs and immediately have a functional monitoring baseline. Then I customize on top - adding environment-specific queries, adjusting alert thresholds, and pinning the dashboards most relevant to my team."

Q33: "Walk me through deploying Operations-Logs."

SAY THIS: "Before deployment, I verify prerequisites: DNS forward and reverse records for the appliance FQDN, NTP server accessibility, a static IP allocation, network connectivity to all log sources, sufficient storage for my retention requirements, and certificate decisions. I deploy through SDDC Manager as a fleet-managed deployment. I provide the FQDN, IP, subnet, gateway, DNS servers, NTP servers, admin password, and deployment size based on environment scale. Deployment takes 30 to 60 minutes. After deployment, I validate: DNS resolution works both directions, the UI loads, admin login works, at least one log source is connected, logs are actively flowing, and the certificate is valid. Then I configure syslog forwarding on ESXi hosts by setting Syslog.global.logHost to the Operations-Logs address, install relevant content packs, and pin the dashboards I need."

Q34: "How do you build a query path that proves root cause using logs?"

SAY THIS: "I follow a three-step query path: symptom, correlation, and proof. Step one, symptom: I search for log entries matching the reported problem. For example, if users cannot log into vCenter, I search for 'authentication failure' or 'login failed' in the last 2 hours filtered to vCenter source. Step two, correlation: I look for related events around the same time. If I find authentication failures starting at 8:43 AM, I search for vCenter service events between 8:30 and 8:45 AM. I might find that the STS service restarted at 8:41 AM. Step three, proof: I dig into what caused the correlated event. I search for errors in the STS service before 8:41 AM and find out-of-memory errors at 8:39 AM. Now I have a provable chain: STS ran out of memory at 8:39, crashed and restarted at 8:41, and users could not authenticate during the restart from 8:41 to 8:45. Each step is backed by timestamped log evidence."

Q35: "What log sources are critical in a VCF environment?"

SAY THIS: "Five critical sources. ESXi hosts: vmkernel.log for hardware, storage, and network events; hostd.log for VM operations; auth.log and shell.log for security auditing. vCenter Server: vpxd.log for core service operations; ssoAdminServer.log for authentication; vpxd-alert.log for critical events. NSX Manager: syslog for network operations; nsxapi.log for API calls; audit logs for security rule changes. SDDC Manager: lifecycle

operation logs for domain management, host commissioning, and update operations. VCF Operations for Logs itself: ingestion health logs to ensure the logging infrastructure is working. I prioritize forwarding in this order: ESXi and vCenter first because they cover the most common troubleshooting and audit scenarios, NSX second for security visibility, SDDC Manager third for lifecycle operations."

Q36: "How do you alert on logs without creating noise?"

SAY THIS: "The principle is alert on symptoms, not on every event. Instead of alerting on every ERROR log line, which generates thousands of alerts, I alert on patterns that indicate a real problem. Three techniques. First, rate-based alerts: alert when the error rate from a single host exceeds 100 errors per hour, which is significantly above the normal baseline of under 10. Second, pattern-based alerts: alert when a specific dangerous pattern appears, like 5 or more authentication failures for the same username within 10 minutes, which suggests brute force. Third, absence-based alerts: alert when a log source stops sending data for more than 15 minutes, which means I have lost visibility. Each alert has a documented playbook entry that tells the responder: what triggered it, how to verify if it is real, and what to do about it."

Q37: "What is your log retention strategy?"

SAY THIS: "I use tiered retention based on log type and compliance requirements. Operational logs like vmkernel and vpxd get 30 days of full detail. These are high-volume but rarely needed beyond the last month for troubleshooting. Security and audit logs like authentication events, shell access, and configuration changes get 90 days to 1 year depending on compliance requirements. HIPAA and SOX environments often require 1 year. Compliance scan results get 1 year minimum for audit evidence. I also differentiate between online and archive retention: the last 30 days are online in Operations-Logs for fast searching, and older data is archived to cheaper storage for compliance retrieval. Storage planning formula: approximately 1 GB per day per 10 hosts for 30-day online retention as a starting estimate, then adjust based on actual volume."

Q38: "How would you investigate a suspected unauthorized access using logs?"

SAY THIS: "Four-step investigation. Step one, identify the scope: search for all authentication events for the suspected user or IP address across all log sources for the last 30 days. Note which systems were accessed, from which source IPs, at what times. Step two, identify anomalies: compare the activity pattern to the user's normal behavior. Unusual source IPs, unusual times like 3 AM access, unusual systems that the user does not normally touch, or privilege escalation events are all flags. Step three, trace actions: for each session identified, search for what the user did after logging in. In vCenter, check task and event logs. In ESXi, check shell.log for commands executed. In NSX, check for firewall rule changes. Step four, document evidence: compile a timeline with log evidence for each finding, including source log name, timestamp, and exact log entry. Preserve the logs by exporting the relevant entries before any retention policy removes them."

Q39: "How do you validate that Operations-Logs is working correctly?"

SAY THIS: "Five validation checks I run weekly. First, ingestion rate: I check the log ingestion dashboard to verify the rate is consistent with the baseline. A sudden drop means a source stopped sending. A sudden spike could mean a log storm from a misbehaving component. Second, source inventory: I verify all expected sources are actively sending. I maintain a source checklist and compare against the connected sources list. Third, latency: I check the time between when a log is generated and when it appears in Operations-Logs. More than a few minutes of delay impacts real-time troubleshooting. Fourth, query performance: I run my standard saved queries and verify they return results in a reasonable time. Slow queries may indicate storage or indexing issues. Fifth, alert function: I verify that log-based alerts are firing correctly by checking the alert history against known events."

Q40: "What would you put on a security-focused log dashboard?"

SAY THIS: "Five widget groups. First row: authentication failure heatmap showing failed logins by time and source, with a count of unique usernames and source IPs. This is my brute force detection surface. Second row: privileged access tracking showing SSH sessions to ESXi hosts, direct console logins, and service account usage. Third row: configuration change audit showing firewall rule modifications, permission changes, and role assignments in vCenter and NSX. Fourth row: certificate and password events showing expiration warnings,

rotation events, and failed certificate validations. Fifth row: log ingestion health for security sources specifically, because if an attacker can disable logging on a system, they can operate without visibility. Each widget has alerting behind it so I am notified of high-risk patterns in real time, not just when I look at the dashboard."

BEHAVIORAL / "TELL ME ABOUT A TIME" (10 Questions)

Q41: "Tell me about a time you had to explain a complex technical issue to a non-technical audience."

SAY THIS: "Our CIO asked why we needed to purchase additional hosts for the production cluster. Instead of showing CPU and memory charts, I built an executive dashboard in VCF Operations showing two things: a capacity runway indicator showing we had approximately 60 days before CPU exhaustion at current growth, and a business impact statement: 'After this date, we cannot onboard the 3 new applications on the project roadmap.' I also showed that right-sizing oversized VMs would buy us 30 additional days but would not eliminate the need for new hardware. By presenting it as a business timeline with a clear decision point rather than a technical metrics discussion, the CIO approved the purchase in the same meeting. The lesson I took away is that executives do not need to understand CPU percentages; they need to understand business risk and timelines."

Q42: "Tell me about a time you reduced alert noise in a monitoring environment."

SAY THIS: "When I took over the VCF Operations environment, there were over 300 active alerts at any given time. The NOC team had developed alert fatigue and was ignoring everything, which meant real issues were being missed. I conducted a two-week audit. I categorized every alert definition into four buckets: actionable as-is, actionable with threshold adjustment, informational only, and not useful. I found that 60% of the noise came from 12 alert definitions with thresholds that were too sensitive for our environment. I adjusted those thresholds based on 30 days of metric history, raising them to above the 95th percentile of normal operation. I disabled 15 alert definitions that had never resulted in human action. After the cleanup, active alert count dropped from over 300 to under 40, and every alert that fired was associated with a runbook entry. Within a month, the NOC team was responding to alerts again because they trusted that each one was meaningful."

Q43: "Tell me about a time you automated a manual process."

SAY THIS: "Every Monday morning, an engineer spent 45 minutes manually pulling capacity data from VCF Operations and formatting it into a weekly report for the ops meeting. I wrote a Python script that authenticates to the VCF Operations API, pulls cluster resource data, calculates capacity remaining and days to exhaustion, identifies oversized VMs and old snapshots, formats everything into a report, and emails it to the team at 6 AM Monday. The script took me about 4 hours to build and test. It has run every week without intervention since deployment. The engineer who used to create the report manually now spends that time on actual operations work. The key design decisions were: credentials stored in environment variables, all operations are read-only, the script is idempotent, and it includes error handling that sends a different email to me if the script fails, so I know to investigate."

Q44: "Tell me about a time you prevented an outage."

SAY THIS: "During my daily checks, I noticed a capacity remaining alert on one of our production datastores. It was at 12% free space and trending down. I investigated and found that a developer had taken a VM snapshot two weeks ago and forgotten about it. The snapshot was growing by approximately 8 GB per day. At that rate, the datastore would have run out of space in about 5 days, which would have caused every VM on that datastore to pause, affecting around 40 production VMs. I contacted the developer, confirmed the snapshot was no longer needed, consolidated it during a maintenance window, and reclaimed 95 GB. I then automated a weekly snapshot report that identifies all snapshots older than 7 days and notifies the VM owners. Since implementing that automation, we have not had another runaway snapshot situation."

Q45: "Tell me about a time you disagreed with a team member's approach."

SAY THIS: "A colleague wanted to set up monitoring alerts that paged the on-call engineer for every vMotion event, arguing that unexpected vMotion could indicate DRS misconfiguration. I disagreed because vMotion is a normal, expected operation in a DRS-enabled cluster, and alerting on every occurrence would create massive noise. Instead, I proposed alerting on the symptom of a DRS problem: if a VM migrates more than 3 times in 1 hour, that could indicate a DRS thrashing condition, which is actually worth investigating. We tested both approaches in a dev environment. My colleague's approach generated over 200 alerts per day. My approach generated zero alerts during normal operation and correctly fired when we simulated a DRS misconfiguration. We went with the symptom-based approach. The lesson was that monitoring should alert on conditions that require human intervention, not on normal operations."

Q46: "Tell me about a time you had to learn a new technology quickly."

SAY THIS: "When our team adopted VCF 9, I needed to get up to speed on VCF Operations quickly because I was the designated monitoring engineer. I built a structured 30-day learning plan. Week 1, I focused on the UI, object model, and building dashboards. Week 2, I went deeper into alerts, capacity, compliance, and role-based dashboards. Week 3, I deployed Operations-Logs and built log dashboards. Week 4, I learned the API, built automation scripts, and conducted mock presentations. Each week had daily objectives with deliverables, and I ran timed incident simulations on Fridays to build muscle memory under pressure. By day 30, I could build and present dashboards, triage issues using a structured runbook, query logs to prove root cause, and automate health reporting via the API. The key was treating learning as a project with milestones, not just reading documentation."

Q47: "Tell me about a time you improved a process."

SAY THIS: "Our incident response process had no standard triage sequence. Different engineers started in different places, and incidents took varying amounts of time based on who was on call. I created a triage runbook with a fixed 15-minute structure: scope assessment in minutes 0-2, alert investigation in minutes 2-5, object trace in minutes 5-8, metric analysis in minutes 8-12, and documentation plus escalation decision in minutes 12-15. I built a VCF Operations dashboard specifically designed to support this workflow, with widgets ordered to match the runbook steps. After training the team and running practice drills, our average initial triage time dropped from 25 minutes to under 15 minutes, and the quality of our incident notes improved because everyone was documenting the same information in the same format."

Q48: "Tell me about a time you handled a high-pressure situation."

SAY THIS: "At 2 AM, I was paged for a P1 alert: a production ESXi host had become unresponsive, and 12 VMs serving customer-facing applications were down. I followed my triage runbook. First, I confirmed the scope in VCF Operations: one host down, 12 VMs affected, no other hosts in the cluster impacted. Second, I verified HA was attempting to restart the VMs on other hosts but some were failing due to insufficient capacity. Third, I manually powered on the highest priority VMs by temporarily suspending two non-critical VMs to free capacity. Fourth, I contacted the hardware vendor because the host failure appeared to be a hardware issue. Within 20 minutes of being paged, 10 of 12 VMs were back online on other hosts. The remaining 2 came back when I freed additional capacity. I documented the full timeline in my RCA and added a capacity buffer recommendation to prevent the HA capacity constraint from recurring."

Q49: "What is your biggest weakness in operations?"

SAY THIS: "Earlier in my career, I would spend too long on root cause analysis during an active incident instead of focusing on service restoration first. I learned that the priority during an incident is restoring service, not understanding why it broke. Now I follow a strict separation: during the incident, I focus exclusively on getting users back to working state. After the incident is resolved, I do the thorough root cause analysis. My triage runbook enforces this by having a decision point at minute 15: 'Can I fix this in the next 15 minutes? If not, what is the fastest path to service restoration while I continue investigating?' This has made me faster at resolving impact even when the root cause takes longer to identify."

Q50: "Why do you want this role?"

SAY THIS: "I want to work in a VCF environment at scale because it combines the three things I am most effective at: building monitoring that tells a story instead of just showing charts, automating operational tasks so the team can focus on improvement instead of repetition, and bringing discipline to operations through structured runbooks, alert taxonomy, and capacity planning. I have built this skill set specifically for VCF Operations - dashboards, API automation, log analysis, and operational storytelling. I want to bring that capability to a team that values operational excellence and gives me the opportunity to make the infrastructure more observable, more automated, and more resilient."

D. WHITEBOARD PROMPTS (WITH COMPLETE SOLUTIONS)

Whiteboard 1: "Design an Ops dashboard for VM performance triage"

DRAW THIS ON THE WHITEBOARD:

```
+-----------------------------+-----------------------------+
|   VM SELECTOR               |   VM HEALTH BADGE           |
|   (Object List - VMs)       |   (Scoreboard - Health)     |
|   [Click a VM to update     |   Shows: Green/Yellow/Red   |
|    all widgets below]       |                             |
+-----------------------------+-----------------------------+
|   CPU USAGE (Line Chart)    |   MEMORY USAGE (Line Chart) |
|   Last 24h, CPU Usage %     |   Last 24h, Memory Active % |
|   + CPU Ready Time overlay  |   + Balloon/Swap overlay    |
+-----------------------------+-----------------------------+
|   DISK LATENCY (Line Chart) |   NETWORK THROUGHPUT        |
|   Last 24h, Read/Write ms   |   Last 24h, TX/RX KBps     |
|   Red line at 20ms          |   + Dropped packets overlay |
+-----------------------------+-----------------------------+
|   RELATED ALERTS (Table)              FULL WIDTH          |
|   Alerts for selected VM, sorted by time, newest first    |
+-----------------------------------------------------------+

INTERACTIONS:
VM Selector → drives all other widgets
Click any VM → Health, CPU, Memory, Disk, Network, Alerts all update

WHY THIS DESIGN:
- Operator selects a VM at the top (one click)
- Immediately sees health + 4 key metrics + related alerts
- Red lines on charts show thresholds (e.g., 20ms disk latency)
- No page navigation needed - everything is on one dashboard
```

SAY THIS WHILE DRAWING: "I design the dashboard around the triage workflow. The operator has a VM name from a user complaint. They select it in the top-left selector. Every widget below updates via interactions. They immediately see health, CPU with ready time overlay, memory with balloon/swap overlay, disk latency, network throughput, and related alerts. The CPU ready time and memory balloon metrics are the ones users actually feel, so I overlay them on the usage charts. I put a red threshold line at 20ms on disk latency because that is where users start noticing storage slowness. Alerts at the bottom provide context for what VCF Operations already knows about this VM."

Whiteboard 2: "Design a log dashboard for auth failures and suspicious activity"

DRAW THIS:

```
+-----------------------------+-----------------------------+
|   AUTH FAILURE HEATMAP      |   FAILURE COUNT BY USER     |
|   (X: Time, Y: Source)      |   (Bar Chart - Top 10)      |
|   Color = failure count     |   Sorted by count           |
+-----------------------------+-----------------------------+
|   FAILURE DETAIL TABLE                FULL WIDTH          |
|   Columns: Timestamp | Username | Source IP | Target |    |
|   Message | Sorted: newest first                          |
+-------------------------------------------------------------+
|   BRUTE FORCE INDICATOR     |   SOURCE IP GEO             |
|   (Scoreboard)              |   (Table - unique IPs)      |
|   "5+ failures same user    |   IP | Country | Failures   |
|    in 10 min = RED"         |                             |
+-----------------------------+-----------------------------+
|   PRIVILEGED ACCESS LOG               FULL WIDTH          |
|   SSH sessions, root logins, service account usage        |
+-------------------------------------------------------------+

ALERTS BEHIND THIS DASHBOARD:
- 5+ failures same user in 10 min → P2 alert
- SSH to ESXi host outside maintenance window → P1 alert
- New source IP for service account → P2 alert
```

Whiteboard 3: "Explain VCF Operations API auth and where Swagger lives"

DRAW THIS:

```
CLIENT (Postman/Python/cURL)
        |
        | 1. POST /suite-api/api/auth/token/acquire
        |    Body: {"username":"admin","password":"***","authSource":"local"}
        v
VCF OPERATIONS API
        |
        | 2. Returns: {"token": "abc123...", "validity": 21600}
        v
CLIENT stores token
        |
        | 3. GET /suite-api/api/alerts
        |    Header: Authorization: OpsToken abc123...
        |    Header: Accept: application/json
        v
VCF OPERATIONS API
        |
        | 4. Returns: JSON alert data
        v
CLIENT processes response
        |
        | 5. POST /suite-api/api/auth/token/release
        |    Header: Authorization: OpsToken abc123...
        v
TOKEN INVALIDATED

SWAGGER UI LOCATION:
https://<ops-fqdn>/suite-api/doc/swagger-ui.html
```

E. MOCK INTERVIEW SCORING RUBRIC

```
SCORING RUBRIC (rate each 1-10)
================================
```

```
1. SIGNAL OVER NOISE                                    /10
   - Does the candidate choose the right metrics to monitor?
   - Can they explain WHY a metric matters, not just name it?
   - Do they filter alerts effectively (P1-P4 taxonomy)?
   Score 8+: Explains metric selection with business impact reasoning
   Score 5-7: Names correct metrics but weak on the "why"
   Score <5: Lists metrics randomly without prioritization

2. ORDER OF OPERATIONS                                  /10
   - Does the candidate triage in a structured sequence?
   - Do they assess scope before diving into detail?
   - Do they check blast radius (Relationships) before acting?
   Score 8+: Has a repeatable triage framework, explains each step
   Score 5-7: Reasonable sequence but some steps out of order
   Score <5: Jumps to conclusions without systematic investigation

3. STORYTELLING                                         /10
   - Can they convert raw metrics into executive narrative?
   - Do they include impact + prevention, not just "what happened"?
   - Can they present a 5-minute briefing coherently?
   Score 8+: Clear timeline, business impact, prevention actions
   Score 5-7: Tells the story but misses impact or prevention
   Score <5: Just describes technical facts without narrative

4. AUTOMATION MATURITY                                  /10
   - Do they automate safely (read-only first, idempotent)?
   - Can they explain when NOT to automate?
   - Do they keep credentials out of code?
   Score 8+: Demonstrates safe automation principles with examples
   Score 5-7: Automates but weak on safety considerations
   Score <5: Scripts without testing or safety considerations

5. DASHBOARD DESIGN                                     /10
   - Are dashboards audience-appropriate (NOC vs Exec)?
   - Do they use widget interactions for drill-down?
   - Is every widget actionable (no decoration)?
   Score 8+: Designs decision dashboards with clear audience targeting
   Score 5-7: Builds functional dashboards but not audience-tuned
   Score <5: Dashboards are just metric displays

6. API COMFORT                                          /10
   - Can they explain token auth flow without hesitation?
   - Do they know where Swagger UI lives?
   - Can they describe a practical API workflow?
   Score 8+: Explains auth + demonstrates API workflow confidently
   Score 5-7: Knows the basics but hesitates on details
   Score <5: Cannot describe the auth flow

7. LOG MASTERY                                          /10
   - Can they pivot from symptoms to log proof?
   - Do they know the critical log sources?
   - Can they build a query path with correlated timestamps?
   Score 8+: Demonstrates a clear symptom → correlation → proof path
   Score 5-7: Can search logs but weak on correlation
   Score <5: Cannot describe a structured log investigation

8. OVERALL CONFIDENCE                                   /10
   - Do they present without hesitation?
   - Are answers structured (not rambling)?
   - Do they admit knowledge gaps honestly?
   Score 8+: Presents fluently, structured answers, honest about gaps
   Score 5-7: Some hesitation but generally competent
   Score <5: Reads from notes or rambles without structure

TOTAL:                                                  /80
70+: Ready for interviews - schedule them
55-69: Need focused practice on weak areas (1-2 more weeks)
```

```
Below 55: Continue the 30-day plan, repeat weeks 3-4
```

VCF OPERATIONS LAB WORKBOOK - COMPLETE HANDS-ON GUIDE

Step-by-Step | Validation Checks | What Could Go Wrong

LAB TRACK A - DASHBOARDS (VCF Operations)

LAB A1: CREATE & CONFIGURE A DASHBOARD

Objective: Create a dashboard from scratch with multiple widgets for cluster health visibility.

Prerequisites:

• VCF Operations instance accessible via browser

• Admin or operator credentials with dashboard create permissions

• At least one vCenter adapter configured and collecting data

Estimated Time: 45 minutes

STEP-BY-STEP:

Step 1: Navigate to Dashboard Creation

1. Open browser, go to: `https://<your-ops-fqdn>/ui`
2. Enter username: `admin` (or your admin account)
3. Enter password: `[your password]`
4. Click **Log In**
5. In the left navigation panel, under **Infrastructure Operations**, click **Dashboards & Reports**
6. You will see dashboard categories: **Favorites**, **Recents**, and **All**. Click the **Manage** tab.
7. Click the **Create** button (look for a "+" icon or "Create Dashboard" button)

Step 2: Name the Dashboard

8. A blank dashboard canvas opens in edit mode
9. At the top of the canvas, find the dashboard name field
10. Click the name field and type: `Lab A1 - Cluster Health Overview`
11. Do NOT save yet - we need to add widgets first

Step 3: Add Widget 1 - Health Scoreboard

12. On the left side of the canvas, find the widget panel (it may say "Widgets" or show a list of widget types)
13. Find the widget type called **Scoreboard** (or **Health Chart**)
14. Click and drag it onto the dashboard canvas - drop it in the top-left area
15. A configuration dialog/panel appears:

• **Title field:** Type `Cluster Health`

• **Description (optional):** Type `Shows health status of all clusters`

• **Object Type / Resource Kind:** Click the dropdown, search for and select `Cluster Compute Resource`

• **Metric:** Click the dropdown, search for and select `Badge|Health` (or `Badge > Health`)

• If there is a "Column" or "Display" option: select `Health` display mode

16. Click **Save** (or **Apply** or **Done** - the exact button depends on your version)
17. The widget should now appear on the canvas showing health badges for your clusters

Step 4: Add Widget 2 - Top VMs by CPU

18. From the widget panel, find **Top-N** (or **Top N Chart**)
19. Drag it onto the canvas to the RIGHT of your first widget
20. Configuration:

• **Title:** Type `Top 10 VMs - CPU Usage`

• **Object Type:** Select `Virtual Machine`

• **Metric:** Select `CPU|Usage (%)` (or `CPU > Usage Percent`)

• **Count / Top N:** Set to `10`

• **Sort:** Descending (highest first)

21. Click **Save/Apply**

Step 5: Add Widget 3 - Active Alerts

22. From the widget panel, find **Alert List** (or **Object Alert List**)
23. Drag it below the first two widgets (bottom-left area)
24. Configuration:

• **Title:** Type `Active Critical & Immediate Alerts`

• **Alert Filters (if available):**

• Status: `Active`

• Criticality: Check `Critical` and `Immediate` (uncheck Warning, Information)

• **Sort by:** Time (newest first)

25. Click **Save/Apply**

Step 6: Add Widget 4 - Capacity Scoreboard

26. From the widget panel, find **Scoreboard** again
27. Drag it to the bottom-right area (next to the Alert List)
28. Configuration:

• **Title:** Type `Cluster Capacity Remaining`

• **Object Type:** Select `Cluster Compute Resource`

• **Metric:** Select `Badge|Capacity Remaining` (or `Summary > Capacity Remaining Percent`)

29. Click **Save/Apply**

Step 7: Arrange Widgets

30. If widgets are overlapping or misaligned:

• Click and drag widget title bars to reposition them

• Drag widget edges/corners to resize them

• Arrange in a 2x2 grid:

• Top-left: Cluster Health

• Top-right: Top 10 VMs - CPU Usage

• Bottom-left: Active Alerts

• Bottom-right: Cluster Capacity Remaining

Step 8: Save the Dashboard

31. Click the **Save** button (disk icon or Save button at the top of the dashboard editor)

32. Wait for confirmation message (usually a brief notification)

33. The dashboard is now saved and visible in the Dashboards & Reports list

VALIDATION CHECKLIST:

```
[ ] Dashboard appears in Dashboards & Reports list with name "Lab A1 - Cluster Health Overview"
[ ] Cluster Health widget displays colored badges (green/yellow/orange/red) for clusters
[ ] Top 10 VMs widget shows VM names with CPU usage percentages
[ ] Alert List widget shows active alerts (or "No alerts" if none exist)
[ ] Capacity Remaining widget shows percentage values for clusters
[ ] All 4 widgets are visible without scrolling (arranged properly)
[ ] Dashboard loads within 10 seconds
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

No data in widgets | Widgets show "No data available" or are empty | Check that vCenter adapter is connected and collecting data. Go to Administration > check adapter/solution status > verify the vCenter adapter shows "Collecting" status. Data may take up to 10 minutes after adapter configuration.

Cannot find widget types | Widget panel does not show expected widget types | You may have a different version. Try searching in the widget panel. Some versions use "Views" instead of "Widgets." Check if you are in edit mode (not view mode).

Permission denied on Create | Create button is grayed out or missing | Your user role does not have dashboard create permissions. Contact your admin to assign the appropriate role (at minimum: Content Admin or PowerUser).

Dashboard does not save | Error message on save | Check if the dashboard name contains special characters. Try a simpler name. Also check if you have exceeded the dashboard limit (if one exists in your environment).

Widgets show wrong data | Data appears but does not match expected objects | Check the Object Type / Resource Kind setting in each widget. If you selected "Host System" instead of "Cluster Compute Resource," you will see host data instead of cluster data. Edit the widget and correct the object type.

Slow dashboard loading | Dashboard takes >30 seconds to render | Reduce the time range (try Last 1 Hour instead of Last 24 Hours). Reduce Top-N count from 10 to 5. Check if VCF Operations appliance is under resource pressure.

JOURNAL PROMPT:

Write down: What was the hardest part of creating this dashboard? What would you add if you had more time? What question does this dashboard answer?

LAB A2: MANAGE DASHBOARDS (CLONE, EDIT, FAVORITES)

Objective: Clone a prebuilt dashboard, edit it, pin favorites, use recents.

Prerequisites: Lab A1 completed. At least one prebuilt dashboard exists in the system.

Estimated Time: 30 minutes

STEP-BY-STEP:

Step 1: Find and Clone a Prebuilt Dashboard

1. Go to **Infrastructure Operations > Dashboards & Reports**

2. Scroll through the list. Look for dashboards that were NOT created by you (prebuilt/system dashboards). Common ones:

• "vSphere VM Utilization"

• "Cluster Performance"

• "Datastore Performance"

• "VM Performance"

3. Click on one to open it (example: "Cluster Performance" or similar)

4. Click the **three-dot menu** (or Actions menu) at the top-right corner of the dashboard

5. Click **Clone** (or **Save As** depending on version)

6. A new dashboard is created. It will be named something like "Cluster Performance - Clone" or "Copy of Cluster Performance"

Step 2: Rename and Edit the Cloned Dashboard

7. Click **Edit** (pencil icon) on the cloned dashboard

8. Change the name to: `NOC - Cluster Performance`

9. Review each widget:

• Are there widgets you do not need? Click the widget, then click the delete/remove icon (trash can) to remove it

• Are there widgets you want to add? Drag new ones from the widget panel

10. Save the dashboard

Step 3: Set Favorites

11. Go back to **Dashboards & Reports list**

12. Find your `Lab A1 - Cluster Health Overview` dashboard in the list

13. Click the **star icon** to the left of or next to the dashboard name

14. The star should change to filled/solid = it is now a Favorite

15. Find your `NOC - Cluster Performance` dashboard

16. Click its **star icon** to favorite it too

17. At the top of the Dashboards & Reports list, look for a **Favorites** filter/tab

18. Click it - you should see ONLY your two favorited dashboards

Step 4: Verify Recents

19. Click back to show ALL dashboards

20. Open 3 different dashboards (click into each, then use the back button or click Dashboards & Reports list again)

21. Look for a **Recents** section or filter

22. Verify the 3 dashboards you just viewed appear there in order of most recent first

Step 5: Manage Dashboard (Share/Export)

23. Go to **Infrastructure Operations > Dashboards & Reports > Manage** (or look for a gear icon)

24. Find your `Lab A1 - Cluster Health Overview` dashboard

25. Select it (checkbox or click)

26. Look for options:

• **Share:** Set sharing to your user group

• **Export:** Download the dashboard as a file (for backup)

• **Delete:** (do NOT delete - just note that the option exists)

27. If Export is available, export it and save the file

VALIDATION CHECKLIST:

```
[ ] Cloned dashboard exists with name "NOC - Cluster Performance"
[ ] Cloned dashboard has been edited (at least one widget added or removed)
[ ] Lab A1 dashboard shows a filled/solid star (favorited)
[ ] NOC dashboard shows a filled/solid star (favorited)
[ ] Favorites filter shows exactly the 2 favorited dashboards
[ ] Recents section shows the dashboards you recently opened
[ ] You found the Share and Export options (even if you did not use them)
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

Clone option missing | No Clone in the menu | Some prebuilt/system dashboards may not be clonable. Try a different dashboard. Or look for "Save As" instead of "Clone."

Cannot favorite | Star icon not visible | Check your permissions. Some read-only roles may not allow favorites. Also, the star may be very small - look carefully next to the dashboard name.

Manage Dashboards not found | No Manage option in the nav | In some versions, Manage is under Infrastructure Operations > Dashboards & Reports > Manage. Or it may be a sub-menu under the Dashboards top-level nav.

LAB A3: CONFIGURE WIDGET INTERACTIONS

Objective: Make a dashboard dynamic by connecting widgets with interactions.

Prerequisites: Lab A1 completed (dashboard with 4 widgets).

Estimated Time: 30 minutes

STEP-BY-STEP:

Step 1: Open Your Dashboard for Editing

1. Go to **Dashboards** > find `Lab A1 - Cluster Health Overview`

2. Click **Edit** (pencil icon)

Step 2: Configure Interaction from Health Scoreboard to VM List

3. Click on the **Cluster Health** widget to select it

4. Look for the **Widget Interactions** option:

• This may be a chain-link icon on the widget toolbar

• Or right-click the widget > **Edit Interactions**

• Or look in the widget's three-dot menu > **Interactions**

5. The Interaction Editor opens. You will see:

• **Source widget:** Cluster Health (the one you selected)

• **Target widgets:** A list or table of other widgets on the dashboard

6. Find the row for **Top 10 VMs - CPU Usage** (your VM list widget)

7. In the interaction dropdown for that target, select **Selected Object** (meaning: when you click a cluster in the source widget, pass that cluster object to the target widget)

8. Find the row for **Active Critical & Immediate Alerts**

9. Set its interaction to **Selected Object** as well

10. Find the row for **Cluster Capacity Remaining**

11. Set its interaction to **Selected Object**

12. Click **Apply** or **Save** to save the interactions

Step 3: Test the Interactions

13. Click **Save** to save the dashboard and exit edit mode

14. You should now be in view mode

15. In the **Cluster Health** widget, click on a specific cluster (click on its name or health badge)

16. Watch the other widgets:

• **Top 10 VMs** should update to show only VMs from the selected cluster

• **Alerts** should update to show only alerts for the selected cluster (or its children)

• **Capacity** should update to show only the selected cluster's capacity

17. Click a different cluster - verify all widgets update again

18. If there is a "clear selection" option (clicking empty space or a reset button), verify that clears the filter and shows all data again

VALIDATION CHECKLIST:

```
[ ] Clicking a cluster in Health widget updates the VM list to show only that cluster's VMs
[ ] Clicking a cluster updates the Alert List to show only that cluster's alerts
[ ] Clicking a different cluster changes all widgets to the new selection
[ ] Selecting "all" or clearing selection restores all widgets to full data view
[ ] Interactions persist after saving and reopening the dashboard
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

No interaction option | Cannot find Widget Interactions in the menu | Make sure you are in edit mode. Some widget types may not support being an interaction source. The Scoreboard widget should support it. Check documentation for your specific version.

Interaction not working | Click a cluster but other widgets do not change | Verify the interaction type is set to "Selected Object" not "Selected Resource" or another type. Also verify the target widgets are configured to accept the object type (Cluster Compute Resource).

Target widget shows "No data" after selection | Widget goes blank when filtering | The target widget may be configured for a different object type that does not relate to clusters. For example, if the alert list is filtered to only

show host alerts, selecting a cluster may not return matching results. Remove the severity filter temporarily to test.

LAB TRACK B - API (VCF Operations)

LAB B1: FIND SWAGGER UI & EXPLORE ENDPOINTS

Objective: Locate API documentation and execute a simple call via Swagger UI.

Prerequisites: VCF Operations instance accessible via browser.

Estimated Time: 30 minutes

STEP-BY-STEP:

Step 1: Open Swagger UI

1. Open a new browser tab
2. Type the URL: `https://<your-ops-fqdn>/suite-api/doc/swagger-ui.html`

• Example: `https://vrops.lab.local/suite-api/doc/swagger-ui.html`

3. Press Enter
4. The page loads showing the VCF Operations REST API documentation
5. You should see a list of API categories/groups

Step 2: Explore the API Categories

6. Scroll through the page. Note each category:

• **Authentication:** Token acquire/release

• **Alerts:** Alert management endpoints

• **Resources:** Inventory/resource endpoints

• **Stats / Statistics:** Metrics data endpoints

• **Dashboards:** Dashboard CRUD endpoints

• **Reports:** Report generation/download

• **Notifications:** Alert notification configuration

• (There may be more categories)

7. Click on the **Authentication** category to expand it
8. You should see endpoints like:

• `POST /api/auth/token/acquire` - Get a token

• `POST /api/auth/token/release` - Release a token

Step 3: Authenticate via Swagger UI

9. Click on `POST /api/auth/token/acquire`
10. Click the **Try it out** button
11. In the request body editor, enter:

```
{
  "username": "admin",
  "password": "YourActualPassword",
```

```
  "authSource": "local"
}
```

12. Click the **Execute** button
13. Scroll down to see the response:

• **Response code:** Should be `200`

• **Response body:** JSON containing a `"token"` field

14. Copy the entire token value (it will be a long string)
15. Save it in a text editor - you will need it for the next steps

Step 4: Make an Authenticated API Call

16. Scroll up to find the **Alerts** category
17. Click to expand it
18. Click on `GET /api/alerts`
19. Click **Try it out**
20. Look for an **Authorization** field or header input
21. Enter: `OpsToken <paste-your-token-here>` (with a space between OpsToken and the token)
22. Click **Execute**
23. Check the response:

• **Response code:** Should be `200`

• **Response body:** JSON containing an `"alerts"` array

24. Scroll through the response - these are real alerts from your environment

Step 5: Try Another Endpoint

25. Find the **Resources** category
26. Click `GET /api/resources`
27. Click **Try it out**
28. Enter the same Authorization header: `OpsToken <your-token>`
29. Optionally add a parameter: `resourceKind = ClusterComputeResource`
30. Click **Execute**
31. Review the response - you should see your cluster objects

VALIDATION CHECKLIST:

```
[ ] Swagger UI page loads at /suite-api/doc/swagger-ui.html
[ ] Token acquired successfully (200 response with token value)
[ ] GET /api/alerts returns 200 with alert data
[ ] GET /api/resources returns 200 with resource data
[ ] You can identify at least 5 API endpoint categories
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

Swagger UI page not found | 404 error | The URL path may differ in your version. Try: `/suite-api/doc/swagger-ui/` (with trailing slash) or `/suite-api/docs/`. Check the VCF Operations documentation for the exact path for your version.

Authentication fails | 401 or 403 response | Check username/password spelling. Verify authSource is correct ("local" for local accounts). Check that the account is not locked. Try logging into the UI with the same credentials

to verify they work.

Token not accepted | 401 on subsequent calls | Make sure you include "OpsToken " (with the space) before the token value. Make sure the token has not expired. Copy the full token string without any extra whitespace or line breaks.

SSL/TLS error | Browser warns about certificate | If using a self-signed certificate, accept the browser warning and proceed. For curl, add the -k flag.

LAB B2: TOKEN AUTH WORKFLOW (POSTMAN)

Objective: Acquire a token and make authenticated calls in Postman.

Prerequisites: Postman installed. VCF Operations instance accessible.

Estimated Time: 45 minutes

STEP-BY-STEP:

Step 1: Install Postman (if not installed)

1. Go to `https://www.postman.com/downloads/`
2. Download the version for your operating system
3. Install and open Postman
4. Create a free account or sign in (you can also skip sign-in and use offline)

Step 2: Create an Environment

5. In Postman, click the **Environments** tab on the left sidebar (or the gear icon at top-right)
6. Click **+** (Create New Environment)
7. Name the environment: `VCF Ops Lab`
8. Add these variables (click "Add new variable" for each):

Variable | Initial Value | Current Value | Type

ops_host | vrops.lab.local | vrops.lab.local | default

ops_user | admin | admin | default

ops_pass | YourPassword | YourPassword | secret

ops_token | | | default

9. Click **Save**
10. In the top-right corner of Postman, select `VCF Ops Lab` from the environment dropdown

Step 3: Create a Collection

11. Click **Collections** tab on the left sidebar
12. Click **+** (Create New Collection)
13. Name the collection: `VCF Ops - Core`
14. Right-click the collection > **Add Folder** > Name: `Auth`
15. Right-click the collection > **Add Folder** > Name: `Alerts`
16. Right-click the collection > **Add Folder** > Name: `Resources`

Step 4: Create the Token Acquire Request

17. Right-click the `Auth` folder > **Add Request**

18. Name the request: `Acquire Token`

19. Change the method dropdown from GET to **POST**

20. In the URL bar, type: `https://{{ops_host}}/suite-api/api/auth/token/acquire`

21. Click the **Headers** tab:

• Add: Key = `Content-Type`, Value = `application/json`

• Add: Key = `Accept`, Value = `application/json`

22. Click the **Body** tab:

• Select **raw**

• Select **JSON** from the format dropdown

• Enter this body:

```
{
    "username": "{{ops_user}}",
    "password": "{{ops_pass}}",
    "authSource": "local"
}
```

23. Click the **Scripts** tab (or **Tests** tab in older Postman versions):

• Click **Post-response** (or Tests)

• Enter this JavaScript code:

```
if (pm.response.code === 200) {
    var jsonData = pm.response.json();
    pm.environment.set("ops_token", jsonData.token);
    console.log("Token saved successfully");
}
```

24. Click **Save** (Ctrl+S)

Step 5: Send the Auth Request

25. Click the blue **Send** button

26. Look at the response panel at the bottom:

• **Status:** Should show `200 OK`

• **Body:** Should show JSON with a "token" field

27. Verify the token was saved: Click the **Environment** quick-look icon (eye icon at top-right) - the `ops_token` variable should now have a value

Step 6: Create and Send an Alerts Request

28. Right-click the `Alerts` folder > **Add Request**

29. Name: `List Active Alerts`

30. Method: **GET**

31. URL: `https://{{ops_host}}/suite-api/api/alerts`

32. Headers:

• Add: Key = `Authorization`, Value = `OpsToken {{ops_token}}`

• Add: Key = `Accept`, Value = `application/json`

33. Click **Save**

34. Click **Send**

35. Response should be `200 OK` with JSON alert data

Step 7: Create and Send a Resources Request

36. Right-click the `Resources` folder > **Add Request**
37. Name: `List Clusters`
38. Method: **GET**
39. URL: `https://{{ops_host}}/suite-api/api/resources?resourceKind=ClusterComputeResource`
40. Headers:

• Add: Key = `Authorization`, Value = `OpsToken {{ops_token}}`

• Add: Key = `Accept`, Value = `application/json`

41. Click **Save**
42. Click **Send**
43. Response should be `200 OK` with cluster resource data

Step 8: Create Token Release Request

44. Right-click the `Auth` folder > **Add Request**
45. Name: `Release Token`
46. Method: **POST**
47. URL: `https://{{ops_host}}/suite-api/api/auth/token/release`
48. Headers:

• Add: Key = `Authorization`, Value = `OpsToken {{ops_token}}`

49. Click **Save**

VALIDATION CHECKLIST:

```
[ ] Postman environment "VCF Ops Lab" created with 4 variables
[ ] Collection "VCF Ops - Core" created with 3 folders
[ ] Acquire Token request returns 200 and saves token to environment
[ ] List Active Alerts request returns 200 with alert JSON
[ ] List Clusters request returns 200 with resource JSON
[ ] Release Token request is saved and ready to use
[ ] Token auto-populates in all requests via {{ops_token}} variable
```

LAB TRACK C - LOGS (VCF Operations for Logs + Analyze)

VCF 9.x CONTEXT: In VCF 9.0, log analysis is integrated into VCF Operations under **Infrastructure Operations > Analyze**. This section provides log search, saved queries, extracted fields, event types, trend analysis, and side-by-side query comparison. It requires the **VCF Operations for Logs** component to be deployed. Log data is standardized to RFC 5424 format.

LAB C1: DEPLOY VCF OPERATIONS FOR LOGS

Objective: Deploy the VCF Operations for Logs component. Understand all required inputs.

Prerequisites:

• VCF Operations console accessible with admin credentials (SDDC Manager UI is deprecated in 9.0)

• DNS records (forward + reverse) created for the Operations-Logs FQDN

• Static IP allocated on a network reachable from all VCF components

• NTP server available

Estimated Time: 2 hours (including deployment wait time)

STEP-BY-STEP:

Step 1: Verify Prerequisites

1. Open a command prompt or terminal

2. Verify DNS forward record:

```
nslookup ops-logs.yourdomain.com
```

Expected result: resolves to your allocated IP (e.g., 10.0.0.50)

If it fails: create the A record in your DNS server before proceeding

3. Verify DNS reverse record:

```
nslookup 10.0.0.50
```

Expected result: resolves to ops-logs.yourdomain.com

If it fails: create the PTR record in your DNS server before proceeding

4. Verify NTP server is reachable:

```
ping ntp.yourdomain.com
```

Expected result: successful ping replies

5. Verify the IP address is not already in use:

```
ping 10.0.0.50
```

Expected result: "Request timed out" (no response = IP is available)

6. Document your inputs:

```
DEPLOYMENT INPUTS:
  FQDN: ops-logs.yourdomain.com
  IP Address: 10.0.0.50
  Subnet Mask: 255.255.255.0
  Gateway: 10.0.0.1
  DNS Server(s): 10.0.0.10
  NTP Server(s): 10.0.0.11
  Admin Password: [your chosen password - minimum 8 chars, complexity required]
  Deployment Size: [Small/Medium/Large based on your host count]
```

Step 2: Log into VCF Operations

7. Open browser: `https://<vcf-ops-fqdn>/ui`

8. Log in with admin credentials

(Note: In VCF 9.0, SDDC Manager UI is deprecated. Fleet and lifecycle operations are now in VCF Operations.)

Step 3: Navigate to Operations for Logs Deployment

9. In VCF Operations, navigate to:

• **Fleet Management** > **Lifecycle Management** and look for the Operations for Logs deployment option

• Or check the **Appliances Health & Management** section on the Launchpad/Home page

(Navigation may vary by version - in some deployments, you may still use a separate deployment workflow)

Step 4: Fill in Deployment Form

10. Enter each field from your documented inputs:

• FQDN: `ops-logs.yourdomain.com`

• IP Address: `10.0.0.50`

• Subnet Mask: `255.255.255.0`

• Gateway: `10.0.0.1`

• DNS Server(s): `10.0.0.10`

• NTP Server(s): `10.0.0.11`

• Admin Password: `[your password]` (enter twice to confirm)

• Deployment Size: Select based on environment:

• **Small:** Up to ~20 ESXi hosts

• **Medium:** Up to ~100 ESXi hosts

• **Large:** 100+ ESXi hosts

• Certificate: Select **Auto-generated** (unless you have a custom CA-signed cert ready)

Step 5: Review and Submit

11. Review ALL entered values on the summary screen

12. Double-check: FQDN matches DNS, IP is correct, passwords match

13. Click **Submit** / **Deploy** / **Finish**

Step 6: Monitor Deployment

14. The deployment task appears in SDDC Manager's task list

15. Wait for it to complete (typically 30-60 minutes)

16. Check progress periodically - look for status updates

17. If it fails, read the error message carefully and check:

• DNS records are correct

• IP is not already in use

• Sufficient resources on the target cluster (CPU, memory, storage)

• Network connectivity from the management cluster to the deployment target

Step 7: Post-Deployment Validation

18. Open a new browser tab: `https://ops-logs.yourdomain.com`

19. The Operations-Logs login page should load

20. Log in with the admin password you set during deployment

21. Verify the main dashboard loads

22. Navigate to log sources or configuration to verify the appliance is ready to receive logs

Step 8: Configure First Log Source (ESXi)

23. If not automatically configured, set syslog forwarding on an ESXi host:

• In vCenter, select an ESXi host

• Go to **Configure** > **System** > **Advanced System Settings**

• Find setting: `Syslog.global.logHost`

• Set value to: `tcp://ops-logs.yourdomain.com:514`

• Click **OK** / **Save**

24. Wait 5 minutes for logs to start flowing

25. In Operations-Logs, check the live log view or ingestion dashboard for incoming data

VALIDATION CHECKLIST:

```
[ ] DNS forward record resolves correctly (nslookup FQDN returns correct IP)
[ ] DNS reverse record resolves correctly (nslookup IP returns correct FQDN)
[ ] VCF Operations UI (Infrastructure Operations > Analyze for log analysis) loads at https://<ops-logs-
fqdn>
[ ] Admin login works with the configured password
[ ] Main dashboard renders without errors
[ ] At least one log source is configured and sending data
[ ] New log entries appear in the real-time/recent log view
[ ] No certificate warnings in the browser (or expected self-signed warning)
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

Deployment fails - DNS error | Error mentions "cannot resolve hostname" | Verify DNS records are correct. From SDDC Manager, run nslookup to test. Both forward and reverse must work.

Deployment fails - IP conflict | Error mentions "IP already in use" | Ping the IP to verify. Check ARP tables. If another device is using it, allocate a different IP.

Deployment fails - resources | Error mentions "insufficient resources" | Check the target cluster has enough CPU (4+ vCPU), Memory (16+ GB), and Storage (500+ GB) for the deployment size.

UI does not load after deploy | Browser timeout or connection refused | Wait 10 more minutes - services may still be starting. Check if the VM is powered on in vCenter. Check if the IP is reachable (ping).

No logs flowing | Ingestion shows 0 events | Verify syslog forwarding is configured on at least one host. Verify the Operations-Logs appliance firewall allows port 514. Check that the syslog protocol matches (TCP vs UDP).

Authentication fails at UI | "Invalid credentials" error | You may have mistyped the password during deployment. Check if there is a default admin password in the documentation. Worst case: redeploy with correct password.

LAB C2: USE PREBUILT CONTENT PACKS

Objective: Install content packs and pin the most useful dashboards.

Prerequisites: Lab C1 completed. Operations-Logs deployed and receiving log data.

Estimated Time: 30 minutes

STEP-BY-STEP:

Step 1: Navigate to Content Packs

1. Log into VCF Operations UI (Infrastructure Operations > Analyze for log analysis)

2. Navigate to **Administration** > **Content Packs** (or **Management** > **Content Packs**)

3. You should see a list of available content packs

Step 2: Install Relevant Content Packs

4. Find the **VMware - vSphere** content pack

5. Click on it to see details (dashboards, alerts, queries included)

6. Click **Install** (or **Enable** if already present but disabled)

7. Wait for installation to complete

8. Find the **VMware - NSX** content pack (if available and NSX is in your environment)

9. Click **Install**

10. Find any **General / Linux / Syslog** content pack

11. Click **Install**

Step 3: Explore Installed Dashboards

12. Go to the **Dashboards** section

13. You should now see new dashboards from the content packs:

• ESXi host log overview

• vCenter events dashboard

• Authentication events

• Error trends

• (Others depending on content pack)

14. Open each one and note:

• What data it shows

• What queries it uses

• Whether it has data (if not, the log source for that dashboard may not be configured)

Step 4: Pin Your Top 10

15. For each of the following dashboards (find the closest match in your environment), add to favorites/pin:

1. ESXi Overview / ESXi Log Summary
2. vCenter Events
3. Authentication Failures / Login Events
4. Error Trends
5. NSX Firewall Events (if available)
6. Storage Errors (if available)
7. VM Power Events (if available)
8. Any certificate-related dashboard
9. SDDC Manager Operations (if available)
10. Ingestion / Log Source Health

16. For each dashboard, click the **star** or **pin** or **favorite** icon

VALIDATION CHECKLIST:

```
[ ] At least 2 content packs installed
[ ] New dashboards visible in the dashboard list
[ ] At least 5 dashboards have data (logs flowing for those sources)
[ ] Favorite/pinned dashboards are saved and accessible via Favorites filter
[ ] You can describe what each pinned dashboard is for
```

LAB TRACK D - AUTOMATION (OpenAPI / SDK / PowerCLI)

LAB D1: GENERATE A POSTMAN COLLECTION FROM OPENAPI

Objective: Import an OpenAPI spec to generate a Postman collection automatically.

Prerequisites: Postman installed. OpenAPI spec file available (JSON or YAML).

Estimated Time: 30 minutes

STEP-BY-STEP:

Step 1: Obtain the OpenAPI Spec

1. Option A - From VCF Operations directly:

• Check if your VCF Operations instance publishes the spec at:

`https://<ops-fqdn>/suite-api/doc/swagger.json` or

`https://<ops-fqdn>/suite-api/doc/openapi.json`

• If accessible, save the file to your computer

2. Option B - From Broadcom Developer Portal:

• Search for "VCF Operations API specification" on the Broadcom developer portal

• Download the OpenAPI 3.0 spec file (JSON or YAML format)

3. Option C - From GitHub:

• Search for VMware/Broadcom VCF API specification repositories

• Download the relevant spec file

Step 2: Import into Postman

4. Open Postman

5. Click the **Import** button (top-left of the Postman window)

6. Either:

• Drag and drop the spec file into the import dialog

• Click **Upload Files** and select the spec file

• Paste a URL to the spec file

7. Postman will analyze the file and show you what it will create

8. You should see: "X endpoints will be imported as a new collection"

9. Click **Import**

Step 3: Explore the Generated Collection

10. In your Collections sidebar, find the newly imported collection

11. Expand it - you should see folders for each API category

12. Expand each folder - you should see individual requests with:

• Pre-configured URLs

• Parameters

• Example request bodies

13. Note: The generated collection will NOT have authentication pre-configured

14. You need to add your environment variables and auth headers (as in Lab B2)

Step 4: Configure Auth for the Generated Collection

15. Right-click the collection > **Edit**

16. Click the **Authorization** tab

17. Set type to **API Key**:

• Key: `Authorization`

• Value: `OpsToken {{ops_token}}`

• Add to: **Header**

18. This sets a default authorization for all requests in the collection

19. Click **Save**

VALIDATION CHECKLIST:

```
[ ] OpenAPI spec file obtained (JSON or YAML)
[ ] File imported into Postman
[ ] Collection created with multiple folders and endpoints
[ ] Authorization configured at the collection level
[ ] At least one endpoint tested successfully (after running token acquire)
```

LAB D2: EXPLORE VCF SDK SAMPLES (PYTHON)

Objective: Locate SDK samples and run a basic example.

Prerequisites: Python 3.8+ installed. pip available.

Estimated Time: 45 minutes

STEP-BY-STEP:

Step 1: Set Up Python Environment

1. Open a terminal / command prompt

2. Create a project directory:

```
mkdir vcf-ops-automation && cd vcf-ops-automation
```

3. Create a virtual environment:

```
python -m venv venv
```

4. Activate the virtual environment:

• Windows: `venv\Scripts\activate`

• Mac/Linux: `source venv/bin/activate`

5. Install the requests library (minimum needed for API calls):

```
pip install requests
```

6. If the VCF SDK is available via pip:

```
pip install vcf-sdk
```

(If this package does not exist or is named differently, proceed with raw requests)

Step 2: Create a Basic Auth Script

7. Create a new file: `hello_vcf_api.py`

8. Type the following code exactly:

```
#!/usr/bin/env python3
"""Basic VCF Operations API authentication test."""
```

```
import requests
import os
import sys
import urllib3

# Suppress SSL warnings for lab (remove in production)
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# Read configuration from environment variables
OPS_HOST = os.environ.get("VCF_OPS_HOST")
OPS_USER = os.environ.get("VCF_OPS_USER")
OPS_PASS = os.environ.get("VCF_OPS_PASS")

if not all([OPS_HOST, OPS_USER, OPS_PASS]):
    print("ERROR: Set these environment variables:")
    print("  VCF_OPS_HOST = your VCF Operations FQDN")
    print("  VCF_OPS_USER = your username")
    print("  VCF_OPS_PASS = your password")
    sys.exit(1)

# Step 1: Acquire token
print(f"Connecting to {OPS_HOST}...")
auth_url = f"https://{OPS_HOST}/suite-api/api/auth/token/acquire"
auth_body = {
    "username": OPS_USER,
    "password": OPS_PASS,
    "authSource": "local"
}

try:
    resp = requests.post(auth_url, json=auth_body, verify=False)
    resp.raise_for_status()
except requests.exceptions.RequestException as e:
    print(f"ERROR: Authentication failed: {e}")
    sys.exit(1)

token = resp.json()["token"]
print("Token acquired successfully.")

# Step 2: Make an API call (list resources)
headers = {
    "Authorization": f"OpsToken {token}",
    "Accept": "application/json"
}

resources_url = f"https://{OPS_HOST}/suite-api/api/resources"
params = {"resourceKind": "ClusterComputeResource", "pageSize": "10"}

try:
    resp = requests.get(resources_url, headers=headers, params=params, verify=False)
    resp.raise_for_status()
except requests.exceptions.RequestException as e:
    print(f"ERROR: API call failed: {e}")
    sys.exit(1)

data = resp.json()
resources = data.get("resourceList", [])
print(f"\nFound {len(resources)} cluster(s):")
for r in resources:
    name = r.get("resourceKey", {}).get("name", "Unknown")
    kind = r.get("resourceKey", {}).get("resourceKindKey", {}).get("resourceKind", "Unknown")
    print(f"  - {name} ({kind})")

# Step 3: Release token
release_url = f"https://{OPS_HOST}/suite-api/api/auth/token/release"
requests.post(release_url, headers=headers, verify=False)
```

```
print("\nToken released. Done.")
```

Step 3: Run the Script

9. Set environment variables:

• Windows:

```
set VCF_OPS_HOST=vrops.lab.local
set VCF_OPS_USER=admin
set VCF_OPS_PASS=YourPassword
```

• Mac/Linux:

```
export VCF_OPS_HOST=vrops.lab.local
export VCF_OPS_USER=admin
export VCF_OPS_PASS=YourPassword
```

10. Run the script:

```
python hello_vcf_api.py
```

11. Expected output:

```
Connecting to vrops.lab.local...
Token acquired successfully.

Found 3 cluster(s):
  - Cluster-Prod-01 (ClusterComputeResource)
  - Cluster-Prod-02 (ClusterComputeResource)
  - Cluster-Dev-01 (ClusterComputeResource)

Token released. Done.
```

VALIDATION CHECKLIST:

```
[ ] Python virtual environment created and activated
[ ] requests library installed
[ ] Script authenticates successfully (token acquired)
[ ] Script lists cluster resources from your environment
[ ] Token is released at the end
[ ] No hardcoded credentials in the script
```

LAB D3: INSTALL VCF POWERCLI & RUN A SMALL SCRIPT

Objective: Install PowerCLI and execute basic inventory commands.

Prerequisites: PowerShell 5.1+ installed (Windows) or PowerShell 7+ (cross-platform).

Estimated Time: 45 minutes

STEP-BY-STEP:

Step 1: Install PowerCLI

1. Open PowerShell as Administrator (right-click > Run as Administrator)

2. Run this command:

```
Install-Module -Name VMware.PowerCLI -Scope CurrentUser -Force -AllowClobber
```

3. If prompted about untrusted repository, type `Y` and press Enter

4. Wait for installation (may take several minutes)

5. Verify installation:

```
Get-Module -Name VMware.PowerCLI -ListAvailable
```

Expected: Shows the VMware.PowerCLI module with version number

Step 2: Configure PowerCLI

6. Run these commands:

```
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false
Set-PowerCLIConfiguration -DefaultVIServerMode Multiple -Confirm:$false
Set-PowerCLIConfiguration -ParticipateInCeip $false -Confirm:$false
```

Step 3: Connect to vCenter

7. Run:

```
Connect-VIServer -Server "vcenter.yourdomain.com" -User "administrator@vsphere.local" -Password "Your
Password"
```

Expected: Connection established message with server name and user

Step 4: Run Basic Commands

8. List all clusters:

```
Get-Cluster | Format-Table Name, HAEnabled, DrsEnabled, @{N='HostCount';E={($_ | Get-VMHost).Count}}
```

9. List all hosts with status:

```
Get-VMHost | Format-Table Name, ConnectionState, PowerState, NumCpu, @{N='MemGB';E={[math]::Round($_.
MemoryTotalGB,1)}}, @{N='MemUsed%';E={[math]::Round($_.MemoryUsageGB/$_.MemoryTotalGB*100,1)}}
```

10. List VMs with snapshots:

```
Get-VM | Get-Snapshot | Format-Table VM, Name, Created, @{N='SizeGB';E={[math]::Round($_.SizeGB,2)}}
```

11. Get total VM count:

```
(Get-VM).Count
```

Step 5: Disconnect

12. Run:

```
Disconnect-VIServer -Confirm:$false
```

VALIDATION CHECKLIST:

```
[ ] PowerCLI module installed and verified
[ ] Configuration set (cert ignore, CEIP off)
[ ] Successfully connected to vCenter
[ ] Get-Cluster returns cluster data
[ ] Get-VMHost returns host data with status
[ ] Get-VM | Get-Snapshot returns snapshot data (or empty if no snapshots exist)
[ ] Disconnected cleanly
```

WHAT COULD GO WRONG:

Problem | Symptom | Solution

Install fails - access denied | "Access to the path is denied" | Run PowerShell as Administrator. Or use -Scope CurrentUser flag.

Install fails - gallery not trusted | Warning about untrusted repository | Type Y to proceed. Or run: Set-PSRepository -Name PSGallery -InstallationPolicy Trusted

Connect fails - cert error | "The SSL connection could not be established" | Run Set-PowerCLIConfiguration -InvalidCertificateAction Ignore first

Connect fails - auth error | "Cannot complete login due to an incorrect user name or password" | Verify credentials. Check if the account is administrator@vsphere.local (not just admin). Try logging into vCenter web UI with same

credentials.

Commands return empty | Get-Cluster returns nothing | Verify you are connected (run $global:DefaultVIServers to check). The connection may have timed out. Reconnect.